

Determinants of Hyperparameter Invariance in Connectome Reservoir Computing

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Miles Walter Churchland

May 2026

Approved for the Division
(Computer Science)

Greg Anderson

Acknowledgements

I am deeply grateful to my advisor, Greg Anderson, for his mentorship, support, and guidance throughout this project. His thoughtful revisions, consistent kindness, and help getting me access to ORCA were invaluable in shaping this work.

I am also deeply grateful to Jordi Garcia-Ojalvo. Our previous work together shaped my interest in both connectomics and reservoir computing, and his mentorship helped make this project possible.

I am grateful to Anna Ritz for suggesting valuable experiments, pushing back on ideas, and supporting me through the inevitable failures that come with modeling complex systems.

Thank you to my partners, Remi Cannon and Autumn Aabram, for putting up with long ramblings about connectomes and reservoir computers. Remi, thank you for your love, humor, and the many long study sessions in the library. Autumn, thank you for your warmth, kindness, and love, which gave me steadiness when this work felt impossible. I am deeply grateful to both of you.

I am also grateful to my friends, who made this process quite joyful: Tara, Sabine, Alta, Enzo, Marley, Rosa, Benny, Kiana, Claire, Nana, Henry, Lumen. Their friendship, humor, and support helped carry me through this thesis.

List of Abbreviations

ANN	Artificial neural network
CAN	Canal-associated neurons (CANL/CANR)
C. elegans	<i>Caenorhabditis elegans</i>
CV	Coefficient of variation
Dale’s law	Principle that a neuron’s outgoing synapses share the same sign
E/I	Excitatory/inhibitory
EI ratio	Excitatory-to-inhibitory ratio
ER	Erdős–Rényi
ESN	Echo state network
GR	Generalization rank
Gym	Gymnasium reinforcement-learning environment toolkit
HPC	High-performance computing
IPC	Information processing capacity
KL	Kullback–Leibler divergence
KR	Kernel rank
L/R	Left/right
LSTM	Long short-term memory
MC	Memory capacity
MG	Mackey–Glass time series
NARMA-N	Nonlinear autoregressive moving average of order N
NRMSE	Normalized root mean squared error
PPO	Proximal policy optimization
RC	Reservoir computing
RNN	Recurrent neural network
SNN	Spiking neural network
SR	Spectral radius ($\rho(W)$)
SW	Small-world
W	Reservoir weight matrix
W^{in}	Input weight matrix
WS	Watts–Strogatz model

Contents

1	Introduction	1
2	Reservoir Computers and Echo State Networks	3
2.1	Reservoir Computers: an Introduction	3
2.2	Echo State Networks	3
2.3	Hyperparameters	4
3	Background	7
3.1	The <i>C. elegans</i> Connectome	7
3.1.1	Connectome	7
3.1.2	Omission of CANL/CANR	8
3.2	Task Agnostic Network Statistics	8
3.2.1	Motivation	8
3.2.2	Kernel Rank	9
3.2.3	Generalization Rank	10
3.2.4	Memory Capacity	11
3.2.5	Information Processing Capacity	13
3.3	Invariance across Hyperparameters	15
4	Methods	17
4.1	Graph Perturbations	17
4.1.1	Connection-Shuffle	17
4.1.2	Weight-Shuffle	17
4.1.3	Spectral Radius Normalization & Scaling	18
4.2	Small-world networks	18
4.2.1	Small-World Topology Enhances the Echo State Property and Signal Propagation	18
4.3	Network Architectures	19
4.3.1	<i>C. elegans</i> connectome	19
4.3.2	Original <i>C. elegans</i> connectome with weight shuffle	19
4.3.3	Original <i>C. elegans</i> connectome with connection shuffle	19
4.3.4	Connection-shuffle with Weight-Shuffle	19
4.3.5	<i>C. elegans</i> architecture with ± 1 weights using original connection signs	19

4.3.6	C. elegans architecture with ± 1 weights and shuffled signs preserving sign balance	19
4.3.7	C. elegans architecture with ± 1 weights under connection shuffling	22
4.3.8	C. elegans architecture with ± 1 weights under connection shuffling and weight shuffling	22
4.3.9	C. elegans architecture with binary weights	22
4.3.10	C. elegans architecture with binary weights and connection shuffle	22
4.3.11	C. elegans Topology with Gaussian Weights	22
4.3.12	C. elegans–Matched Erdős–Rényi Model with Gaussian Weights	23
4.3.13	C. elegans–Matched Watts–Strogatz Model with Gaussian Weights	23
4.3.14	C. elegans architecture with half Gaussian weights using original connection signs	23
4.3.15	C. elegans architecture with uniform weights using original connection signs	24
4.4	Trial definition	24
4.5	Hyperparameter Sweeps	24
4.6	Experimental Pipeline	25
4.7	KL divergence to a fitted shifted half-normal target	25
4.8	Equivalence testing	26
5	Results	27
5.1	Sign structure and weight distribution shape invariance across architectures	27
5.2	Invariance to degree-preserving topology changes	28
5.3	Half-normal weight perturbations have limited effect on invariance	30
5.4	Global neurotransmitter balance shapes the performance–invariance tradeoff	34
6	Discussion	39
6.1	Future directions	40
7	AI Usage Disclosure	43
8	Supplemental information	45
8.1	Neuroscience Terms Explained for Computer Scientists	45
8.2	Equivalence testing	46
8.3	Tables	46
	Bibliography	51

List of Tables

8.1	Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity.	47
8.2	Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank.	47
8.3	Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity for shuffling experiment 1. .	48
8.4	Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 1.	48
8.5	Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity for shuffling experiment 2. .	48
8.6	Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 2.	49
8.7	Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 3.	49
8.8	Changes in mean CV across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is not preserved in this version of the experiment.	49
8.9	Changes in mean metric value across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is not preserved in this version of the experiment.	50
8.10	Changes in mean CV across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is preserved in this version of the experiment. . .	50
8.11	Changes in mean metric value across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is preserved in this version of the experiment. . .	50

List of Figures

2.1	A basic echo state network architecture. W^{out} edges are trainable. W^{in} and W^{res} are fixed after the network architecture is created and are not updated during training.(Zheng et al., 2020)	5
4.1	Network visualizations for architectures A–H. In all panels, nodes represent neurons, orange edges are excitatory synapses, blue edges are inhibitory synapses, and darker edges indicate stronger synaptic weights. A: original <i>C. elegans</i> connectome. B: original connectome with shuffled weights. C: connection-shuffled <i>C. elegans</i> connectome. D: connection-shuffled connectome with an additional weight shuffle. E: original topology with signed binary (± 1) weights determined by the original edge signs. F: panel E after sign shuffling. G: connection-shuffled version of panel E. H: panel G after sign shuffling.	20
4.2	Additional control architectures I–O. I: <i>C. elegans</i> connectome with binary weights. J: binary-weight <i>C. elegans</i> connectome after connection shuffling. K: <i>C. elegans</i> topology with Gaussian weights. L: <i>C. elegans</i> -matched Erdős–Rényi network with Gaussian weights. M: small-world network with $p = 0.1$ and Gaussian weights. N: sign-preserving Gaussian model. O: sign-preserving uniform model.	21
5.1	Invariance dispersion across architectures	29
5.2	Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the original <i>C. elegans</i> weight matrix and its shuffled variants. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Blue: original <i>C. elegans</i> connectome; dark orange: weight shuffle; light orange: connection shuffle; red: connection+weight shuffle.	31
5.3	Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the <i>C. elegans</i> topology with ± 1 weights and its shuffled variants. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Blue: baseline <i>C. elegans</i> topology with ± 1 weights; gray: weight shuffle; pink: connection shuffle; purple: connection+weight shuffle.	32

5.4	Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the <i>C. elegans</i> topology with binary weights and its degree-preserving connection-shuffled counterpart. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Turquoise: baseline binary-weight network; green: connection-shuffled binary-weight network.	33
5.5	Effect of perturbing nonzero weight magnitudes toward a half-normal distribution while preserving connectivity and edge sign. Starting from the <i>C. elegans</i> network, nonzero weights are transformed as $W'_{ij} = \text{sign}(W_{ij}) W_{ij} + \Lambda Z_{ij} $, $Z_{ij} \sim \mathcal{N}(0, 1)$, where Λ controls perturbation strength. The x-axis shows the KL divergence between the empirical distribution of $ W $ and a fitted shifted half-normal target on a $\log_{10}$ scale; increasing Λ decreases this divergence. Top row: mean metric value versus KL divergence. Bottom row: mean CV versus KL divergence. Colored trajectories correspond to GR (green), KR (red), IPC (orange), and MC (blue). Left and right panels show the same data from two viewing angles. The transparent planes indicate the unperturbed baseline, corresponding to $\Lambda = 0$	35
5.6	IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the original connectome using $\pm$ weights. Increasing sign flip percentage shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. A total of 250 trials were performed for each fraction.	36
5.7	IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the ER random graph using binary weights. Increasing sign flip percentage shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. A total of 250 trials were performed for each fraction.	37
5.8	IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the absolute value of the <i>C. elegans</i> connectome. The black point represents the negative sign percentage observed in the original connectome. A total of 250 trials were performed for each fraction.	37

Abstract

Biological neural circuits maintain functional computation despite substantial variation in synaptic strengths, membrane properties, and inputs. Reservoir computing provides a useful artificial framework for studying this robustness: input signals are projected into a high-dimensional space by a fixed nonlinear dynamical system, called the reservoir. Unlike biological circuits, however, these systems can be highly sensitive to hyperparameters such as spectral radius, input scaling, and leak rate. This thesis asks which structural features of a network support invariance to those parameter changes. Using the *C. elegans* connectome as a biologically grounded reservoir, I measure computational capability with four task-agnostic metrics—Memory Capacity (MC), Information Processing Capacity (IPC), Kernel Rank (KR), and Generalization Rank (GR)—and define hyperparameter invariance as the coefficient of variation of these metrics across sweeps of spectral radius, input scaling, and leak rate.

To isolate the structural determinants of invariance, I construct controlled perturbations that separately modify connectivity topology, weight-magnitude distributions, and excitatory/inhibitory sign structure while preserving the remaining properties. Across these experiments, the empirical connectome consistently occupies a relatively low-variance regime. Preserving global sign structure recovers much of this invariance even when weight magnitude heterogeneity is removed, whereas perturbing sign balance produces the largest changes in both mean performance and hyperparameter sensitivity.

The dominant pattern is a performance–invariance tradeoff. Moving the reservoir toward greater excitatory/inhibitory balance generally increases mean computational performance but also increases variability under hyperparameter change. Topological perturbations more strongly affect memory-based metrics (MC and IPC), while sign-preserving weight-magnitude perturbations more strongly affect kernel-based metrics (KR and GR). These results suggest that hyperparameter invariance in connectome reservoirs emerges from the interaction of global sign balance, weight magnitude distribution, and topological structure.

Dedication

For my parents, Anne Churchland and Michael Brodesky, for my grandmother, Patricia Churchland, whose support, kindness, and love of beauty in scientific inquiry made this work possible.

Chapter 1

Introduction

In 1966, the mathematician and psychologist Seymour Papert initiated the “Summer Vision Project,” an early attempt to develop programs capable of performing image-classification tasks. The project revealed that visual perception was far more difficult to formalize computationally than early researchers had anticipated, and that heuristic approaches were insufficient for the problem. (Nilsson, 2010)

Earlier, in 1962, Hubel and Wiesel studied the visual cortex of cats and found neurons that responded selectively to oriented bars of light at specific locations, as well as more complex cells that responded to such features across a range of positions. These results suggested that biological vision achieves robust computation through layered feature extraction. (Lindsay, 2020)

Fukushima drew directly on these biological principles in 1980 when he developed the Neocognitron, a multilayer neural architecture capable of image classification that was robust to positional shifts. The Neocognitron is widely regarded as a precursor to modern convolutional neural networks. This historical arc is illustrative: when computational principles proved inadequate, progress came from studying biological systems and abstracting structural principles from them. (Fukushima, 1980)

The same logic motivates the present work. Biological neural networks must function reliably across noisy, variable, and changing conditions, making them natural models of robust computation. The gap between artificial and biological robustness is still visible in many domains: engineered systems often remain brittle outside their training regime (Geirhos et al., 2020; Recht et al., 2019), whereas animals routinely generalize behavior across unfamiliar environments (Burden et al., 2024; Pearson, 2000).

This thesis turns to a biological network not to explain the organism itself, but to study what its architecture may reveal about robust computation. Thus, the *C. elegans* connectome is used as a network prior to investigate which of its structural features contribute to hyperparameter-invariant computation.

Chapter 2

Reservoir Computers and Echo State Networks

2.1 Reservoir Computers: an Introduction

Picture a pool of water, with no waves on it—this is a computational reservoir, our computational substrate. Now imagine dropping a stone into the pool—our input—and observing the ensuing waves—our output. One could use the pool to perform computational tasks by dropping multiple stones into it in sequence.¹ Alternatively, one could measure the computational power of the pool without regard to a specific task; this is what this work explores. In this analogy, the input sequence corresponds to the sequence of stones, the nonlinear high-dimensional expansion corresponds to the pool dynamics, and the output sequence corresponds to an observer interpreting the evolving wave patterns. Echo state networks are a class of reservoir computers in which the reservoir state is uniquely determined by the input history.

2.2 Echo State Networks

Echo state networks are discrete-time recurrent reservoir computers, introduced by Jaeger (Jaeger, 2001). Due to their recurrence, echo state networks are particularly useful in time series analysis (Lukoševičius et al., 2012) and chaotic system prediction problems (Jaeger & Haas, 2004). For readers unfamiliar with reservoir computing, we recommend the tutorial-style introduction (Inoue et al., 2026) and the accompanying interactive RC Bootcamp notebooks. (“RC bootcamp”, 2026)

Reservoir computers have d input units, N internal network (more colloquially known as reservoir network or reservoir) units, and p output units. These units are most commonly nodes of a network.² Reservoir computers have three weight matrices: input weights $W^{\text{in}} \in \mathbb{R}^{N \times d}$, internal weights $W^{\text{res}} \in \mathbb{R}^{N \times N}$, and output weights $W^{\text{out}} \in \mathbb{R}^{p \times N}$ (Jaeger, 2001). The reservoir acts as a nonlinear expansion and

¹This has actually been done in *Pattern Recognition in a Bucket* (Fernando & Sojakka, 2003)

²We must say units here, as researchers implement reservoir computing with all sorts of computational media, such as water (Fernando & Sojakka, 2003) and plants (Pieters et al., 2022).

as a memory of the input (Lukoševičius, 2012).

Formally, let $u_t \in \mathbb{R}^d$ be input, $x_t \in \mathbb{R}^N$ the reservoir state, and $y_t \in \mathbb{R}^p$ the output. With leak $\alpha \in (0, 1]$:

$$\tilde{x}_{t+1} = \tanh(W^{\text{res}}x_t + s_{\text{in}}W^{\text{in}}u_{t+1} + b + \xi_t), \quad (2.1)$$

$$x_{t+1} = (1 - \alpha)x_t + \alpha\tilde{x}_{t+1}, \quad (2.2)$$

$$y_t = W^{\text{out}}x_t, \quad (2.3)$$

where $\tanh$ is elementwise, b is an optional bias term and is not included in our model, ξ_t is optional i.i.d. noise and is not included in our model, W^{in} is input weights scaled by s_{in} , $W^{\text{res}} \in \mathbb{R}^{N \times N}$ is in our case sparse, and $W^{\text{out}} \in \mathbb{R}^{p \times N}$ is trained. The parameter p is set to 3 for IPC (Section 3.2.5) to solve for different orders in a single batch; in all other cases, it is set to 1.

Training a network is the process of tuning its parameters to optimize it for a specific task or set of tasks. When training these networks we train solely W^{out} , as opposed to training all weights as is typically done in artificial neural networks. Training exclusively the output weights preserves the dynamics of the reservoir while keeping training computationally inexpensive. When training these networks we “tap” into the dynamics of the “reservoir” by linearly combining different neuron states. Training the reservoir’s weights would reshape the internal dynamics, often reducing the diversity of activity patterns and thereby decreasing the number of linearly independent features available to the readout layer.

The literature emphasizes the importance of the internal network satisfying the echo state property (Jaeger, 2001; Stepney, 2024). The echo state property states that asymptotically, the state of a reservoir $x(n)$ is uniquely determined by the input history u_t , rather than by the initial state of the network x_0 . An elucidating example of this property is a house with a thermostat. Picture two identical houses with the same target temperature u_t and thermostat program W . Imagine house B starts at a cold temperature x_0 and house A starts at a hot temperature x'_0 . With time and a reasonable thermostat program, both temperature states x_n, x'_n will converge and the temperature state x_n will be determined solely by the temperature target.

A reservoir that does not satisfy the echo state property is generally unsuitable for computation. Without the echo state property, the reservoir state is not uniquely determined by the input history and instead depends largely on the initial condition. In practice, this means that recurrent feedback dominates the dynamics: internal activity grows in magnitude and persists independently of the input; the effect of new inputs is overshadowed. As a result, the reservoir no longer functions as a reliably input-driven dynamical system and instead tends toward self-sustaining or unstable feedback patterns.

2.3 Hyperparameters

Echo state networks, like most machine learning models, rely on hyperparameters—parameters specified prior to training that determine global properties of the model

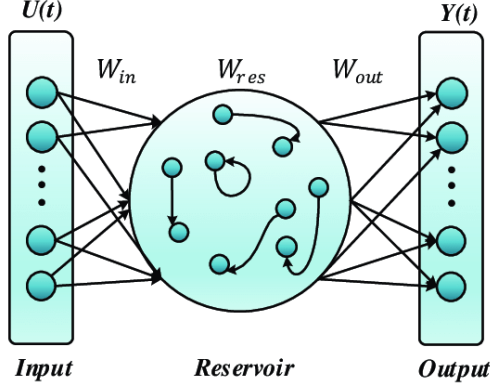


Figure 2.1: A basic echo state network architecture. W^{out} edges are trainable. W^{in} and W^{res} are fixed after the network architecture is created and are not updated during training. (Zheng et al., 2020)

and influence how its neurons behave. In reservoir computing, these parameters do not affect W^{out} directly; instead they influence the intrinsic behavior of the reservoir itself, shaping recurrent feedback, external input, and temporal integration.

Spectral radius $\rho(W^{\text{res}})$: Let $W \in \mathbb{R}^{N \times N}$ be a square matrix with eigenvalues $\{\lambda_i\}_{i=1}^N$. The *spectral radius* of W is defined as

$$\rho(W) = \max_{1 \leq i \leq N} |\lambda_i|.$$

The spectral radius roughly controls the strength of recurrent feedback. Choosing $\rho(W^{\text{res}}) < 1$ promotes the echo state property by making the reservoir dynamics contractive. Values near $\rho(W^{\text{res}}) \approx 1$ typically produce richer, longer-lasting transients while often still preserving the echo state property. Values with $\rho(W^{\text{res}}) > 1$ are less common, since they often lead to reservoirs that do not satisfy the echo state property. However, if the reservoir is driven by sufficiently strong inputs, reservoirs with $\rho(W^{\text{res}}) > 1$ can still satisfy the echo state property. This is due to the nonlinearity of the activation function, in our case tanh, which reduces the effective gain of the recurrence and damps feedback loops. (Lukoševičius, 2012)

Leak rate α : The leak rate controls the effective timescale of the reservoir by interpolating between the previous state and the newly computed activation. The effective recurrence is

$$x_{t+1} \approx [(1 - \alpha)x_t + \alpha\tilde{x}_{t+1}],$$

Smaller α increases the persistence of past inputs and extends temporal memory of the reservoir. Larger α causes the system to update more rapidly, reducing memory but increasing responsiveness. Thus, while the spectral radius controls the strength of recurrence, the leak rate controls how quickly that signal dissipates over time.

Input scaling s_{in} : Input scaling determines the magnitude of the input weights. When input scaling is small, the influence of external input on the reservoir state is reduced relative to that of recurrent feedback. Increasing input scaling drives neurons into the nonlinear regime of the activation function, increasing computational

richness. However, excessive scaling saturates the activation function, reducing sensitivity to inputs and collapsing the effective dimensionality of the reservoir’s state matrix.

Noise ξ_t : Noise introduces stochastic perturbations to the reservoir state. Noise insertion has been shown to improve attractor stability in echo state networks by preventing trajectories from collapsing into undesired fixed points or oscillatory modes, though this comes at the cost of reduced prediction precision (Jaeger, 2001). Because the present experiments focus on task-agnostic structural invariance rather than optimizing performance on a specific task, noise is omitted from our study.

Chapter 3

Background

3.1 The *C. elegans* Connectome

3.1.1 Connectome

A connectome is a description of the neural wiring of an organism. It encodes which neurons are connected, the direction of each connection, and often additional biological metadata such as the number of synaptic terminals and the neurotransmitter associated with each connection.

In this work, we treat the connectome as the foundation for a graph: each neuron is represented as a node, and each synapse as a directed, weighted edge. The connectome used in our study is sourced from the c302 project (Gleeson et al., 2018), which is an initiative to model the neural network of the nematode *C. elegans*. The network is composed of several key components:

Neurons : The database includes all 302 identified neurons in the *C. elegans* nervous system. Our model includes 299 neurons; CANL, CANR, and VC6 are omitted (see Sec. 3.1.2).

Synaptic connections : The c302 connectome model contains 3672 synaptic connections, but 564 of them are neuromuscular junctions; they send neurotransmitters to muscle cells and are not included in our graph. This leaves us with 3108 synaptic connections, categorized into various types based on the nature of the synapse, including chemical synapses and gap junctions. Specifically, we assign inhibitory synapses (GABAergic) negative weights (190 synapses) and excitatory synapses—acetylcholine (394 synapses) and glutamate (962)—positive weights. All remaining transmitters (1562 synapses) are assigned positive sign. This sign assignment is a *working assumption* used to define an excitatory/inhibitory partition for the invariance experiments; it does not attempt to capture receptor-specific or context-dependent effects (e.g., cases where acetylcholine or glutamate can be inhibitory). Because this study examines how coarse sign structure and weight statistics influence task-agnostic reservoir metrics across hyperparameter sweeps, incorporating detailed synapse-type dynamics is both beyond the scope of the analysis and computationally infeasible at the

scale of experiments required. We retain the non-GABA synapses (rather than dropping them) to preserve the observed network density and recurrence, which strongly influence reservoir dynamics and the metrics studied here. This modeling choice undoubtedly affects our results and should be considered when interpreting them; it is discussed further in our conclusion section 5. The connections are represented as directed edges in the network, indicating the direction of signal transmission. Gap junctions are considered to be two distinct edges with independent weights for computational simplicity.

Connection count (weight proxy) : The dataset reports only the number of terminal contacts that neuron i makes onto neuron j , not graded synaptic strengths. We therefore use that contact count c_{ij} as an initial weight, $W_{ij} = c_{ij}$. Treating contact number as a proxy weight is standard in *C. elegans* modeling (Casal et al., 2020; Gleeson et al., 2018) and provides the biologically informed prior for our training.

3.1.2 Omission of CANL/CANR

In the c302 wiring used here, CANL and CANR form gap-junction contacts only with excretory tissues and have no neuronal synapses. Because our node set includes neurons but not excretory cells, including CANL/CANR would produce two isolated vertices (no neuronal in- or out-edges and no neuromuscular outputs). Additionally, VC6 forms an outgoing connection to a muscle, but has no incoming or outgoing synapses to or from other neurons. Nodes with no incoming or outgoing synaptic connections are dynamically inert and therefore do not influence dynamics of the network. Formally, if neuron i satisfies $W_{ij} = 0$ and $W_{ji} = 0$ for all j , then its state does not contribute to the state update of any other neuron. Consequently, adding or removing such isolated nodes does not alter the dynamics of the remaining network. They would still have an internal signal influenced by the input. This internal signal provides no additional useful features to the readout layer, since its temporal activity is linearly dependent with respect to the four neurons with in-degree zero already present in the network.¹ The inclusion or omission of these isolated nodes is therefore irrelevant in this model.

3.2 Task Agnostic Network Statistics

3.2.1 Motivation

What makes a given neural circuit computationally powerful? More precisely, how can two network architectures be compared across a suite of tasks without explicitly training them on those tasks? In reservoir computing, this question is addressed through task-agnostic statistics that quantify intrinsic dynamical properties of the network. Measures such as information processing capacity (IPC), kernel rank (KR), generalization rank (GR), and memory capacity (MC) characterize functional dimensions,

¹Importantly, these four neurons have non-zero out-degree and thus influence network dynamics.

including nonlinearity, separability, generalization, and fading memory. Additionally, these statistics are widely used as benchmarking tools for reservoir networks (Pilati et al., 2026).

3.2.2 Kernel Rank

Returning to the earlier pool analogy, in which input sequences are modeled as stones dropped into water and reservoir states as the resulting wave patterns, we can use this picture to build intuition for the task-agnostic network statistics considered in this work. Imagine dropping stones of different sizes into the pool, one after another, in the same place each time, and repeating this process for many input sequences. Each sequence produces a pattern of waves, corresponding to a reservoir state. Kernel rank is a measure of how distinctly the pool responds to different sets of inputs; if many input sequences lead to the same wave patterns, then the kernel rank will be low. If those input sequences instead map to distinct wave patterns, then the kernel rank will be high.

Kernel rank measures how effectively inputs are projected into a high-dimensional state space so they become linearly separable by a readout W^{out} , serving as a measurable proxy for kernel quality and the diversity of nonlinear transformations created by the reservoir (Stepney, 2024). Higher is better for this metric. Specifically, kernel rank is a measure of the rank of the $n \times m$ matrix whose columns are the internal states $x_{u_i}(t_0)$ at some fixed time t_0 for the preceding input streams u_i , where n is the number of units in the circuit and m is the number of input streams. This is the original definition (Maass et al., 2004); we use a modified version that is also found in the literature (Stepney, 2024). In our implementation, we use the state-matrix variant common in benchmarking: we drive the reservoir with a single i.i.d. input sequence u_t —rather than m input streams—and collect the post-washout² state matrix $X \in \mathbb{R}^{T \times N}$, where each row is the reservoir state at each time step. Kernel rank is computed using effective rank (Roy & Vetterli, 2007b). Because the readout is linear, neurons with very similar activity patterns provide redundant information. We want a kernel whose neurons’ activity patterns vary greatly, such that this can be utilized by our readout. Thus, we need a different measure of rank, since matrix rank only detects exact linear dependence; neurons with highly similar but not identical activity patterns will still increase the rank. We want a measure of rank that captures variance across orthogonal modes. This leads us to effective rank. To compute effective rank, we perform the following steps:

First, we center our values with respect to time:

$$X_c = X - \bar{X}$$

²A washout period is an initial set of timesteps discarded so the reservoir moves away from its arbitrary starting state before the states used for training and testing are collected.

where

$$\bar{X} = \bar{X}_{tj} = \frac{1}{T} \sum_{t'=1}^T X_{t'j}$$

Here, $\bar{X} \in \mathbb{R}^{T \times N}$ is the matrix obtained by repeating, in each row, the mean activity of each neuron over time. Subtracting $\bar{X}$ from X therefore centers each neuron's activity across time. We then get the singular values from the singular value decomposition:

$$S = [\sigma_1, \sigma_2, \dots, \sigma_n] = \text{SVDVALS}(X_c)$$

The values of a singular value decomposition represent the scaling in each orthogonal dimension. When we compute the σ s, we do so by obtaining the square root of the eigenvalues of $X_c^T X_c$. $X_c^T X_c$ is the unnormalized covariance matrix of X_c . A large magnitude at index i, j in $X_c^T X_c$ means that neuron i 's activity is closely aligned with neuron j 's. A low value indicates a weak linear correlation between their signals.

The eigenvalues describe how much variance lies along each orthogonal activity pattern. If many neurons behave similarly, most variance lies in a small number of directions and only a few eigenvalues are large. If neurons exhibit diverse activity patterns, variance spreads across more directions and more eigenvalues become significant.

Next, we ensure that there are no σ values equal to 0 by setting all zero values to ϵ , where $\epsilon = 1e - 12$.

$$S'_i = \max(\epsilon, S_i)$$

We now convert the singular values into a probability distribution by normalizing them by their sum. This yields a distribution over the p_i suitable for computing Shannon entropy.

$$p_i = \frac{S'_i}{\sum_{k=1}^N S'_k}$$

We invoke Shannon entropy to quantify how much information, on average, is needed to encode our singular values. If there is one large singular value and the rest are close to 0, then we need substantially less information to encode the distribution than if there are multiple singular values with similar sizes.

$$H = - \sum_{i=1}^N (p_i \ln p_i)$$

We then compute e^H to obtain the effective rank of our matrix.

$$\text{erank} = e^H$$

3.2.3 Generalization Rank

Returning to the pool, suppose we wanted to observe how much the wave patterns change if we used slightly larger or smaller rocks. We could alter the size of our rocks

slightly and drop them sequentially into the same location in the pool. By comparing the resulting wave pattern to the wave pattern of the unaltered rocks, we can measure how many independent wave patterns the pool produces from these nearly identical inputs. If small changes in the stones lead to many distinct wave patterns, the pool is highly sensitive and maps to a high-dimensional set of states; if small changes lead to similar wave patterns, the pool generalizes and collapses to a lower-dimensional set of states. This is measured by the Generalization Rank.

Generalization Rank (GR) quantifies the sensitivity of reservoir states to small perturbations in the input. It is computed as the effective rank of the difference between state trajectories driven by clean and noise-perturbed inputs. Lower GR indicates that input perturbations induce lower-dimensional deviations in state space, reflecting a reduced sensitivity to noise. Our unperturbed input is

$$u_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(-1, 1)$$

Our perturbed input is

$$u'_t = u_t + \kappa \varepsilon_t, \quad \varepsilon_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

With κ set to 0.01, we run the network on the perturbed u' and collect the states over time X' . We also run the network on the unperturbed input u and collect the states over time X .

$$\text{Generalization Rank} = \text{erank}(X' - X)$$

If the network generalizes well, small input perturbations produce only low-dimensional changes in the reservoir state after the washout period.

3.2.4 Memory Capacity

Returning to the pool analogy, suppose we wish to infer the size of a stone after it has been dropped into the water. The stone itself is no longer visible, but its presence is encoded in the resulting wave pattern. By observing the waves, one could estimate the size of the stone. If multiple stones are dropped sequentially, their wave patterns overlap and interfere. The older a stone, the more its influence dissipates and mixes with subsequent wave patterns. Memory capacity measures how well the current reservoir state can *reconstruct* past inputs at varying delays.

Memory capacity, introduced by Jaeger (Jaeger, 2002), measures how much linear memory a reservoir retains about past inputs. Given an input sequence $u(t) \sim \mathcal{U}(-1, 1)$, the reservoir is trained to reconstruct delayed versions of that input, denoted $\hat{u}_k(t)$, where k is the delay. A high value of MC_k indicates that the reservoir state at time t contains enough information to accurately recover the input from time $t - k$. Jaeger defines memory capacity as

$$\text{MC}_k = \frac{(\text{cov}(u(t - k), \hat{u}_k(t)))^2}{\text{var}(u) \text{var}(\hat{u}_k)}$$

$$MC = \sum_{k=1}^{\infty} MC_k.$$

MC_k measures the correlation between the predicted signal and the original signal or how well the network “remembers” the original signal at delays k . Summing over k gives us how well the network can remember a signal over all delays k . As k increases the MC_k typically decreases. This is intuitive, as we are in a network that has the echo state property and thus the state of the network is determined by its input history and not by its initial state.³ For each delay k , we train a linear readout from the reservoir state $\mathbf{x}(t)$ to reconstruct the delayed input $u(t - k)$.

We follow Jaeger’s definition of MC , but evaluate each delayed reconstruction using the squared Pearson correlation between the target input sequence and the reservoir’s prediction. This form is algebraically equivalent to Jaeger’s covariance–variance ratio, while avoiding the explicit computation of separate sample covariance and variance terms. In the equations below, Y denotes the true delayed input sequence, X denotes the predicted delayed input sequence, and $\epsilon = 1 \times 10^{-12}$ is added to the denominator for numerical stability.

$$\begin{aligned} MC_k &= r_{\text{score}}(X, Y)^2 \\ &= \frac{(\text{cov}(X, Y))^2}{\text{var}(X) \text{var}(Y)} \\ &= \frac{\left(\frac{1}{T-1} \sum_{i=1}^T (X_i - \bar{X})(Y_i - \bar{Y}) \right)^2}{\left(\frac{1}{T-1} \sum_{i=1}^T (X_i - \bar{X})^2 \right) \left(\frac{1}{T-1} \sum_{i=1}^T (Y_i - \bar{Y})^2 \right)} \\ &= \left(\frac{\sum_{i=1}^T (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\left(\sum_{i=1}^T (X_i - \bar{X})^2 \right) \left(\sum_{i=1}^T (Y_i - \bar{Y})^2 \right) + \epsilon}} \right)^2. \end{aligned}$$

We define Ridge regression as follows

$$\text{Ridge_Regression}(X, Y, \xi) = ((X)^T X + \xi I)^{-1} (X)^T Y$$

Notation.

- $X_{d:T_{tr}}^{\text{train}}$: reservoir states on training data from timestep d to the end.
- $X_{d:T_{te}}^{\text{test}}$: reservoir states on test data from timestep d to the end.
- $u_{0:T_{tr}-d}^{\text{train}}$: training inputs from the start up to the end minus a delay d .
- $u_{0:T_{te}-d}^{\text{test}}$: test inputs from the start up to the end minus a delay d .

³Jaeger also shows that for i.i.d. random input, $MC \leq N$ where N is the number of reservoir units in the graph. (Jaeger, 2002; Stepney, 2024)

- I : the identity matrix of size $N \times N$
- ξ : user-specified parameter denoting the strength of L2 regression. We used 0.0001.

In our experiments, we approximate memory capacity by summing this same per-delay score over a finite delay window:

$$\text{MC} = \sum_{d=1}^{D_{\max}} \text{MC}_d = \sum_{d=1}^{D_{\max}} r_{\text{score}}(Y_d, \hat{Y}_d)^2,$$

where

$$Y_d = u_{0:T_{te}-d}^{\text{test}}, \quad \hat{Y}_d = X_{d:T_{te}}^{\text{test}} \text{ Ridge_Regression}(X_{d:T_{tr}}^{\text{train}}, u_{0:T_{tr}-d}^{\text{train}}, \xi).$$

Here $D_{\max}$ is the maximum delay included in the sum; in our experiments, we use $D_{\max} = 30$. This departs from the formal definition of memory capacity, which sums over all delays, but an infinite sum is not computationally feasible. Furthermore, MC_k decays as $k \rightarrow \infty$, so terms at sufficiently large delays contribute negligibly.

To validate this truncation, we swept $D_{\max} \in \{30, 50, 80\}$ across 14 models used in our experiments, using 20 random seeds per model at fixed hyperparameters ($\rho = 0.95$, leak = 0.8, input scaling = 1.0), for 280 evaluations total. We treat $D_{\max} = 80$ as the reference condition. Across these evaluations, $D_{\max} = 30$ captured an average of 100.00%⁴ of the memory capacity measured at $D_{\max} = 80$, with a standard deviation of 0.02%, a minimum of 99.71%, and a 5th percentile of 100.00%. The corresponding model values were in perfect agreement across all 14 models, with Pearson $r = 1.0000$.

We therefore use $D_{\max} = 30$ throughout, since it preserves essentially the full observed memory capacity while being more computationally efficient than larger cutoffs. The results also suggest that this cutoff could likely be reduced further without materially affecting the approximated memory capacity.

3.2.5 Information Processing Capacity

Returning to our pool, suppose we are no longer interested in recovering the size of individual stones, but instead in recovering combinations of stones. When multiple stones are dropped into the pool, their waves do not simply add; they interfere, overlap, and mix. The resulting wave pattern contains not only information about each stone individually, but also information about their interactions. Information Processing Capacity measures how many independent combinations of past stones simultaneously exist in the current wave pattern.

Notation.

- $X_{d:T_{tr}}^{\text{train}}$: reservoir states on training data from timestep d to the end.

⁴This value rounds to 100.00%. The underlying mean was slightly below 100%, as reflected by the nonzero standard deviation and minimum.

- $X_{d:T_{te}}^{\text{test}}$: reservoir states on test data from timestep d to the end.
- $u_{0:T_{tr}-d}^{\text{train}}$: training inputs from the start up to the end minus a delay d .
- $u_{0:T_{te}-d}^{\text{test}}$: test inputs from the start up to the end minus a delay d .
- P_k is a function applying an elementwise Legendre polynomial of degree k .

Let

$$Y_{d,k} = P_k(u_{0:T_{te}-d}^{\text{test}}), \quad \hat{Y}_{d,k} = X_{d:T_{te}}^{\text{test}} \text{ Ridge_Regression}(X_{d:T_{tr}}^{\text{train}}, P_k(u_{0:T_{tr}-d}^{\text{train}}), \xi).$$

Information Processing Capacity measures the ability of the reservoir state $x(t)$ to reconstruct nonlinear functions of delayed input sequences. We start with a uniformly sampled noise sequence of length T on the interval $[-1, 1]$:

$$u_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(-1, 1)$$

We define information processing capacity (IPC) as

$$\text{IPC} = \sum_{d=1}^{D_{\max}} \sum_{k \in K} \max(0.0, r_{\text{score}}(Y_{d,k}, \hat{Y}_{d,k})^2),$$

In the literature, IPC is often summed over all Legendre orders and over a large delay horizon. We truncate this computation to orders $K = \{1, 3, 5\}$ and maximum delay $D_{\max} = 30$, since higher-order and longer-delay contributions are small and our experiments require tens of thousands of IPC evaluations.

We validated the degree truncation by comparing $K = (1, 3, 5)$ against IPC computed over degrees 1 through 10, across 14 models used in our experiments, using 20 random seeds per model at fixed hyperparameters ($\rho = 0.95$, leak = 0.8, input scaling = 1.0), for 280 evaluations total. IPC(1, 3, 5) captured an average of 99.08% of the full IPC, with standard deviation 1.74%, minimum 90.58%, and 5th percentile 94.97%. Agreement with IPC(1, ..., 10) was also extremely high, with Pearson correlation $r = 0.9995$. Thus, restricting our computation to orders 1, 3, and 5 preserves the IPC value to a high degree.

We similarly validated the delay cutoff by computing IPC with $K = \{1, 3, 5\}$ and $D_{\max} \in \{30, 50, 80\}$, treating $D_{\max} = 80$ as the reference. Across the same hyperparameters, models, and number of seeds per model, $D_{\max} = 30$ captured an average of 100.00% of the IPC measured at $D_{\max} = 80$, with standard deviation 0.03%, minimum 99.51%, and 5th percentile 100.00%. Model-wise agreement with the $D_{\max} = 80$ reference was perfect, with Pearson $r = 1.0000$ for both $D_{\max} = 30$ and $D_{\max} = 50$.

Thus, we use IPC(1, 3, 5) with $D_{\max} = 30$ throughout. In our setting, this truncation preserves essentially all measured IPC while substantially reducing computational cost.

3.3 Invariance across Hyperparameters

Variance measures the degree of spread in a data set from its mean value. In this study, we measure variation in task-agnostic statistics (GR, KR, IPC, MC) under hyperparameter change. While the values of these task-agnostic statistics are themselves meaningful—for example, high IPC can indicate a useful network architecture—we are also interested in how these values change under hyperparameter variation across different architectures, rather than considering the values alone. Hence, we measure the coefficient of variation rather than raw variance, since CV provides a measure of variation across different scales.

$$CV = \frac{\text{std}(a)}{\bar{a}}$$

where a denotes the set of values of the network statistic of interest—such as IPC, MC, GR, or KR—across the evaluated hyperparameter settings. It is desirable to have a low CV, as this indicates that the network statistic is invariant to hyperparameter change, meaning the network can be trained with a wide range of hyperparameters without changing the network’s ability to perform a given task.

Chapter 4

Methods

To determine which topological and weight-related features influence robustness, we construct a series of controlled network variants derived from the *C. elegans* connectome. Each architecture is designed to isolate a specific property of the biological network—such as topology, weight distribution, or sign structure—while holding other properties fixed. By comparing the computational behavior of these modified networks to the original connectome, we can assess which features are responsible for observed variation across hyperparameter settings.

4.1 Graph Perturbations

4.1.1 Connection-Shuffle

We define a connection shuffle as a directed degree-preserving edge swap. Let W be the weighted adjacency matrix. We randomly select two existing directed edges (a, b) and (c, d) , where a, b, c, d are distinct and the proposed edges (a, d) and (c, b) do not already exist. We then form a new matrix W' by setting

$$W'_{ad} = W_{ab}, \quad W'_{cb} = W_{cd}, \quad W'_{ab} = 0, \quad W'_{cd} = 0,$$

with all other entries unchanged. In this operation, the weights are carried with the original edges as they are reassigned to new targets. This preserves each node's in-degree and out-degree while shuffling the neighborhood structure.

4.1.2 Weight-Shuffle

Fix the synaptic connectivity pattern and randomly permute the weights assigned to existing connections. Let

$$NZ = \{(i, j) : W_{ij} \neq 0\}$$

denote the set of nonzero entries in the weight matrix. We extract the corresponding weights into a vector,

$$w_{NZ} = \{W_{ij} : (i, j) \in NZ\},$$

randomly shuffle this vector using `numpy.random.shuffle`, and then reassign the shuffled weights back to the same nonzero indices. Thus, the set of existing connections is unchanged, but the weights assigned to those connections are redistributed across the fixed network topology.

For this shuffle, the probability that a given connection changes sign is 11.47%. This corresponds to an expected 356.769 sign changes.

4.1.3 Spectral Radius Normalization & Scaling

Spectral radius normalization and rescaling is done by scaling all weights in our weight matrix by $\frac{\rho_{target}}{\rho(W)}$. This is applied as the final step before running the architectures. Thus, normalization/scaling occur after any weight or sign perturbations.

4.2 Small-world networks

The small-world phenomenon refers to the observation that the distance between two randomly chosen nodes in a network is surprisingly small, even when the network contains many nodes (Barabási, 2016). Small-world networks are characterized by the coexistence of two structural properties: a high clustering coefficient and a short characteristic path length.

4.2.1 Small-World Topology Enhances the Echo State Property and Signal Propagation

Kawai et al. (Kawai et al., 2019) demonstrate that networks with small-world structure (Watts & Strogatz, 1998) ($p = 0.1$) exhibit enhanced memory capacity in reservoir computing when input and output nodes were spatially segregated. They localized input nodes in one region of the network and output nodes in a separate distant region. Small-world networks achieved greater total memory capacity across different spectral radii than networks that did not exhibit small-world properties.

Small-world networks maintained strong performance across a broader range of spectral radii. In contrast, the performance of fully random reservoirs fell sharply outside of a narrow band of spectral radii. Though, notably, fully random networks ($p = 1$) achieved the single highest peak memory capacity—at a spectral radius slightly below one

This result provides conceptual motivation for the present work: a biologically inspired architecture with small-world structure can exhibit comparatively stable computational performance across spectral radii. This is notable because random connectivity is often used in reservoir computing to promote dynamical diversity and high task performance. However, performance and hyperparameter invariance are distinct: the architectures that achieve the highest computational scores may also be the most sensitive to parameter changes. The biological network’s stability therefore does not contradict the usefulness of random reservoirs, but instead suggests a tradeoff between performance and robustness.

4.3 Network Architectures

4.3.1 C. elegans connectome

This uses the biological network and biological weights. Used as a comparison against other methods.

$$W_{ij} = W_{ij}^{cel},$$

4.3.2 Original *C. elegans* connectome with weight shuffle

Starting from the original *C. elegans* connectome, this architecture preserves biological topology but shuffles the weights across existing connections using the weight-shuffle procedure in Section 4.1.2.

4.3.3 Original *C. elegans* connectome with connection shuffle

Starting from the original *C. elegans* connectome, this architecture applies the degree-preserving connection-shuffle procedure in Section 4.1.1.

4.3.4 Connection-shuffle with Weight-Shuffle

First shuffle the connections of the *C. elegans* connectome using Section 4.1.1, then shuffle the weights of the resulting weight matrix using Section 4.1.2. This method is used as a control to show the effects of an additional weight shuffle after a connection shuffle.

4.3.5 C. elegans architecture with ± 1 weights using original connection signs

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). It sets the weight of every connection to 1 or -1 depending on the neurotransmitter.

$$W'_{ij} = \begin{cases} \text{sign}(W_{ij}) & \text{if } W_{ij} \neq 0 \\ 0 & \text{if } W_{ij} = 0 \end{cases}$$

4.3.6 C. elegans architecture with ± 1 weights and shuffled signs preserving sign balance

This control keeps the biological connectivity fixed but randomizes which edges carry inhibitory signs. Starting from the signed binary network in Section 4.3.5, let

$$N_- = \#\{(i, j) : W'_{ij} = -1\}$$

denote the number of negative entries. A random subset of size N_- of nonzero entries of $|W'|$ is assigned sign -1 , and all remaining nonzero entries are assigned sign

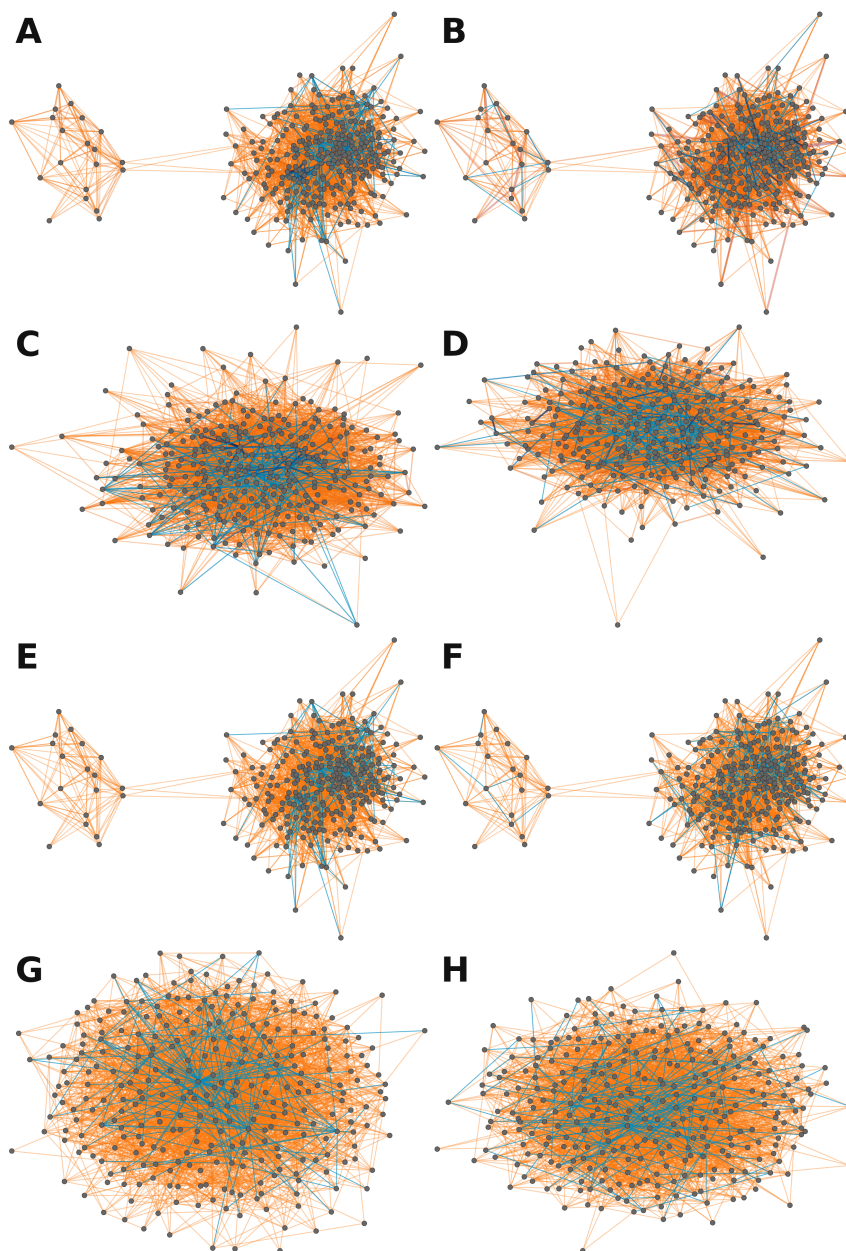


Figure 4.1: Network visualizations for architectures A–H. In all panels, nodes represent neurons, orange edges are excitatory synapses, blue edges are inhibitory synapses, and darker edges indicate stronger synaptic weights. A: original *C. elegans* connectome. B: original connectome with shuffled weights. C: connection-shuffled *C. elegans* connectome. D: connection-shuffled connectome with an additional weight shuffle. E: original topology with signed binary (± 1) weights determined by the original edge signs. F: panel E after sign shuffling. G: connection-shuffled version of panel E. H: panel G after sign shuffling.

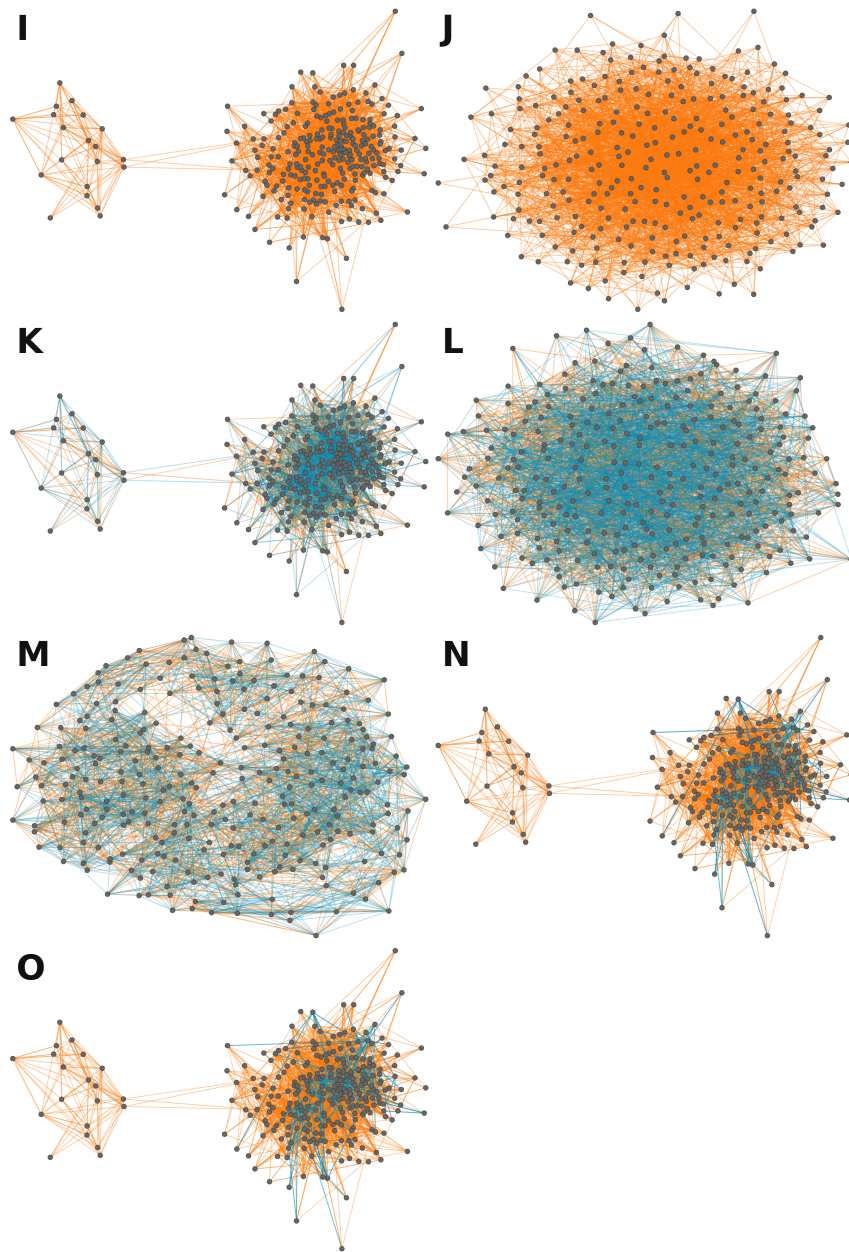


Figure 4.2: Additional control architectures I–O. I: *C. elegans* connectome with binary weights. J: binary-weight *C. elegans* connectome after connection shuffling. K: *C. elegans* topology with Gaussian weights. L: *C. elegans*-matched Erdős–Rényi network with Gaussian weights. M: small-world network with $p = 0.1$ and Gaussian weights. N: sign-preserving Gaussian model. O: sign-preserving uniform model.

+1. This preserves the original excitatory/inhibitory balance while shuffling the sign associated to each edge. Shuffling the signs is equivalent to a weight shuffle in this setting.

4.3.7 C. elegans architecture with ± 1 weights under connection shuffling

Start with 4.3.5. The resulting ± 1 connectome is then connection-shuffled as described in 4.1.1.

4.3.8 C. elegans architecture with ± 1 weights under connection shuffling and weight shuffling

Start with 4.3.5. The resulting ± 1 connectome is then connection-shuffled as described in 4.1.1, and a weight shuffle is subsequently applied to the shuffled matrix as described in 4.1.2. This condition serves as a control for testing whether an additional weight shuffle has any effect once the ± 1 network has already undergone connection shuffling.

4.3.9 C. elegans architecture with binary weights

This ablation begins with the biological *C. elegans* connectivity pattern and sets each weight as follows:

$$W'_{ij} = \begin{cases} 1 & \text{if } W_{ij} \neq 0 \\ 0 & \text{if } W_{ij} = 0 \end{cases}$$

4.3.10 C. elegans architecture with binary weights and connection shuffle

This ablation begins with the biological *C. elegans* connectivity pattern and sets each weight as follows:

$$W'_{ij} = \begin{cases} 1 & \text{if } W_{ij} \neq 0 \\ 0 & \text{if } W_{ij} = 0 \end{cases}$$

The resulting binary connectome is then connection-shuffled as described in 4.1.1.

4.3.11 C. elegans Topology with Gaussian Weights

This control keeps the biological connectivity pattern of the *C. elegans* connectome, but replaces the weight on each existing connection with an independent Gaussian sample:

$$W'_{ij} = \begin{cases} X_{ij} & \text{if } W_{ij} \neq 0 \\ 0 & \text{if } W_{ij} = 0, \end{cases} \quad X_{ij} \sim \mathcal{N}(0, 1).$$

4.3.12 C. elegans–Matched Erdős–Rényi Model with Gaussian Weights

As a random-graph baseline, we constructed networks using the Erdős–Rényi (ER) model. In this model, each possible edge between a pair of nodes is independently included with probability p .

To match the scale of the *C. elegans* connectome, we generated graphs with $N = 299$ nodes. Each potential edge was included independently with probability $p = 0.1$. Because this procedure produces a random number of edges, we subsequently adjusted the edge count to match that of the connectome. Specifically, edges were randomly removed until the total number of connections equaled that of the *C. elegans* network. Removing an edge was implemented by setting the corresponding weight to zero. After fixing the topology, weights were independently assigned to the remaining edges by sampling from a Gaussian distribution.

4.3.13 C. elegans–Matched Watts–Strogatz Model with Gaussian Weights

To test whether small-world organization affects task-agnostic invariance, we used a directed Watts–Strogatz-like control adapted from (Watts & Strogatz, 1998). The graph has the same number of neurons and synaptic connections as the *C. elegans* connectome: $N = 299$ nodes and 3108 directed edges.

The construction begins with a directed ring lattice. Since $3108 = 299 \times 10 + 118$, every node is assigned 10 local outgoing edges, and 118 randomly chosen nodes receive one additional local outgoing edge. This gives exactly 3108 edges before rewiring.

Each outgoing edge is then rewired independently with probability $p = 0.1$. Rewiring changes the target of an edge while preserving its source, so each node’s out-degree is unchanged. New targets are chosen to avoid self-loops and duplicate edges. Thus, the final graph preserves the exact edge count of the biological network while introducing a small-world-like topology. After fixing the topology, weights were independently assigned to the remaining edges by sampling from a Gaussian distribution.

4.3.14 C. elegans architecture with half Gaussian weights using original connection signs

Start with Section 4.3.5. Then reassign the edge weights to random weights sampled from a half-Gaussian distribution centered at 0 with standard deviation 1.

$$W'_{ij} = \text{sign}(W_{ij})(|\mathcal{N}(0, 1)|),$$

4.3.15 C. elegans architecture with uniform weights using original connection signs

Starting from the signed binary network in Section 4.3.5, this control preserves each edge’s original sign while replacing its magnitude with an independent draw from a uniform distribution on $[0, 1]$:

$$W'_{ij} = \text{sign}(W_{ij})X_{ij}, \quad X_{ij} \sim \mathcal{U}(0, 1).$$

4.4 Trial definition

A single trial of a model consists of:

1. A reservoir instantiation for each trial, reset between hyperparameter combinations
2. A performance evaluation for each hyperparameter combination, using the following parameters:
 - **Sequence length** = 2500
 - **Washout time steps** = 500
 - **Perturbation standard deviation** = 0.01 (for calculating GR)
 - **Train time steps** = 1500
 - **Test time steps** = 500
 - **Memory capacity max delay** = 30
 - **Information processing capacity max delay** = 30
 - **Information processing capacity max polynomial order** = 1,3,5
 - **Ridge alpha** = 0.0001 (controls the strength of L2 regularization for ridge regression)

4.5 Hyperparameter Sweeps

We evaluated network performance across a full factorial sweep of spectral radius, leak rate, and input scaling. To compute our coefficient of variation (CV) metric, each trial evaluates a given network’s performance across all combinations of the following hyperparameters:

- Spectral radius: $sr \in \{0.6, 0.8, 0.95, 1.05\}$
- Leak rate: $\text{leak} \in \{0.6, 0.8, 1.0\}$
- Input scaling: $\text{input scaling} \in \{0.1, 0.5, 1.0, 1.5\}$

The coefficient of variation (CV) is computed across hyperparameter combinations within a trial. Each trial uses a single fixed input sequence shared across all hyperparameter combinations to ensure comparability. Reservoir instantiations are independent across trials: for each trial, the reservoir is reconstructed with a unique seed. Each reservoir instantiation in a trial uses the same input sequence; different trials use different input sequences.

Number of trials for histogram experiment 5.1: 500 per model. Total metric evaluations per model: Trials $\times$ hyperparameter grid = 24,000. Number of trials for Figures 5.6, 5.7, and 5.8: 250 per point. Total metric evaluations per point: 12,000. Number of trials for Figure 5.5: 50 per point. Total metric evaluations per point: 2,400.

4.6 Experimental Pipeline

For each trial:

1. Instantiate reservoir computer (nodes and edges)
2. Assign weights/signs
3. Scale to target spectral radius
4. Create an i.i.d. input sequence
5. Run the network on the input sequence and split states into washout/train/test sets
6. Compute task-agnostic metrics (KR, GR, IPC, MC)
7. Repeat for each combination of hyperparameters in our hyperparameter array

Analysis of Trials:

1. Compute CV across hyperparameter grid
2. Average over trials
3. Run equivalence tests and plot results

4.7 KL divergence to a fitted shifted half-normal target

For the weight-perturbation experiment, we quantify changes in the nonzero weight-magnitude distribution using a binned KL divergence between the empirical distribution of $|W|$ and a fitted shifted half-normal target. The magnitudes of finite nonzero weights are shifted so that the minimum value is 0 and then partitioned into 64 equal-width bins over the observed range. The target distribution is fit by setting the

location parameter to the minimum observed magnitude and the scale to the standard deviation of the shifted magnitudes. KL divergence is then computed between the empirical bin frequencies and the corresponding model bin probabilities, after clipping empty-bin probabilities by a small constant (1×10^{-12}) and renormalizing. Smaller values indicate closer agreement with the fitted shifted half-normal target.

4.8 Equivalence testing

We use two one-sided tests (TOST) to test whether the difference in mean CV between architectures lies within equivalence bounds. Δ is calculated independently across experiments and metrics. Equivalence bounds are defined as:

$$\Delta = \eta \cdot \text{median}(CV_{\text{all}})$$

We test:

$$H_0 : |\mu_1 - \mu_2| \geq \Delta$$

$$H_1 : |\mu_1 - \mu_2| < \Delta$$

Equivalence is concluded if both one-sided tests reject H_0 at $\eta = 0.05$.

Chapter 5

Results

Our modeling of the *C. elegans* connectome exhibits consistently low dispersion across hyperparameter sweeps, indicating that its computational properties remain stable under variation in spectral radius, leak rate, and input scaling. This is visible in Fig. 5.1, where the connectome’s coefficient-of-variation (CV) distributions for MC, IPC, KR, and GR are concentrated toward lower values relative to several perturbed architectures. We find that the interaction between sign and edge structure with the distribution of weight magnitudes determines the invariance of computational properties.

5.1 Sign structure and weight distribution shape invariance across architectures

Figure 5.1 shows the coefficient of variation (CV) of each model across hyperparameter sweeps. Lower CV corresponds to greater invariance. The original *C. elegans* connectome lies among the lower-dispersion architectures, whereas the Gaussian-weight architectures without preserved biological sign structure—the *C. elegans*+Gaussian, ER+Gaussian, and WS+Gaussian models—occupy a distinctly higher-CV regime. This separation suggests that invariance depends on more than topology alone; preserving some combination of biological sign structure and constrained weight statistics is important.

Pairwise equivalence testing, seen in tables 8.1 and 8.2, sharpens this interpretation. The two sign-preserving ± 1 models, one that preserves the original edge signs exactly and one that preserves only the global sign balance while randomizing sign placement, are both equivalent to the original connectome for IPC, KR, and GR, but not for MC. Collapsing all nonzero weights to ± 1 therefore preserves much of the connectome’s invariance profile. Moreover, the global sign-balance-preserving model is equivalent to the local sign-preserving model across these metrics, which suggests that exact edge-by-edge sign placement is less important than maintaining the overall excitatory/inhibitory balance. For IPC, KR, and GR, sign structure matters more than the detailed distribution of weight magnitudes.

The binary base model, which preserves topology but removes both sign structure

and weight heterogeneity, also falls in the low-CV cluster and has among the smallest mean CV values across metrics. However, Table 8.1 shows that it is equivalent to the original connectome only for IPC. Thus, although homogeneous weights decrease CV, they do not reproduce the invariance profile of the original connectome.

The sign-preserving Gaussian and sign-preserving uniform models show a different pattern. These two models are equivalent to one another for MC, IPC, and GR, but not for KR, and each is equivalent to the original connectome only for MC. Thus, once topology and sign are fixed, replacing the empirical magnitude distribution with either Gaussian or uniform samples produces similar behavior across several metrics, but neither reproduces the full biological invariance profile. This indicates that weight-magnitude distribution still matters, especially for KR and, relative to the original connectome, for IPC and GR. At the same time, these are the only models equivalent to the connectome for MC, which suggests that MC depends more strongly on retaining heterogeneous weights than do the other metrics. This question is examined further in Section 5.3.

Finally, the ER+Gaussian and WS+Gaussian models are equivalent to one another across all four metrics, suggesting that small-world structure alone does not strongly affect invariance in this setting. The *C. elegans*+Gaussian model joins this group only partially: it is equivalent to WS+Gaussian for MC and GR, and to ER+Gaussian for GR. Once empirical sign structure and weight magnitudes are discarded, topology still influences invariance, with IPC and KR showing the strongest topological dependence.¹

Taken together, these comparisons suggest a layered picture of invariance across architectures. Global excitatory/inhibitory balance is sufficient to recover much of the connectome’s low-variance behavior for IPC, KR, and GR, whereas MC appears to depend more strongly on heterogeneous weights. Topology is not irrelevant, but its effect becomes most apparent after biological sign structure and weight statistics are removed. The invariance profile of the *C. elegans* reservoir is therefore not explained by any single coarse architectural property, but by the interaction of sign balance, weight heterogeneity, and topology.

5.2 Invariance to degree-preserving topology changes

To test whether hyperparameter invariance depends on connectivity itself or on the weights assigned to that connectivity, we consider three degree-preserving shuffling experiments. Each experiment holds node degree fixed but changes a different combination of topology, sign placement, and weight magnitude. Together, they separate the effect of rewiring from the effect of redistributing weights across edges.

The first experiment, shown in Fig. 5.2, Table 8.4, and Table 8.3, uses the original real-valued *C. elegans* weight matrix. In Fig. 5.2, all three shuffled conditions lie to the right of the unshuffled connectome, indicating higher CV and therefore lower

¹Importantly for Section 5.2, one cannot move from the *C. elegans* topology to a WS or ER topology using only degree-preserving weight shuffles, because these models have different degree distributions.

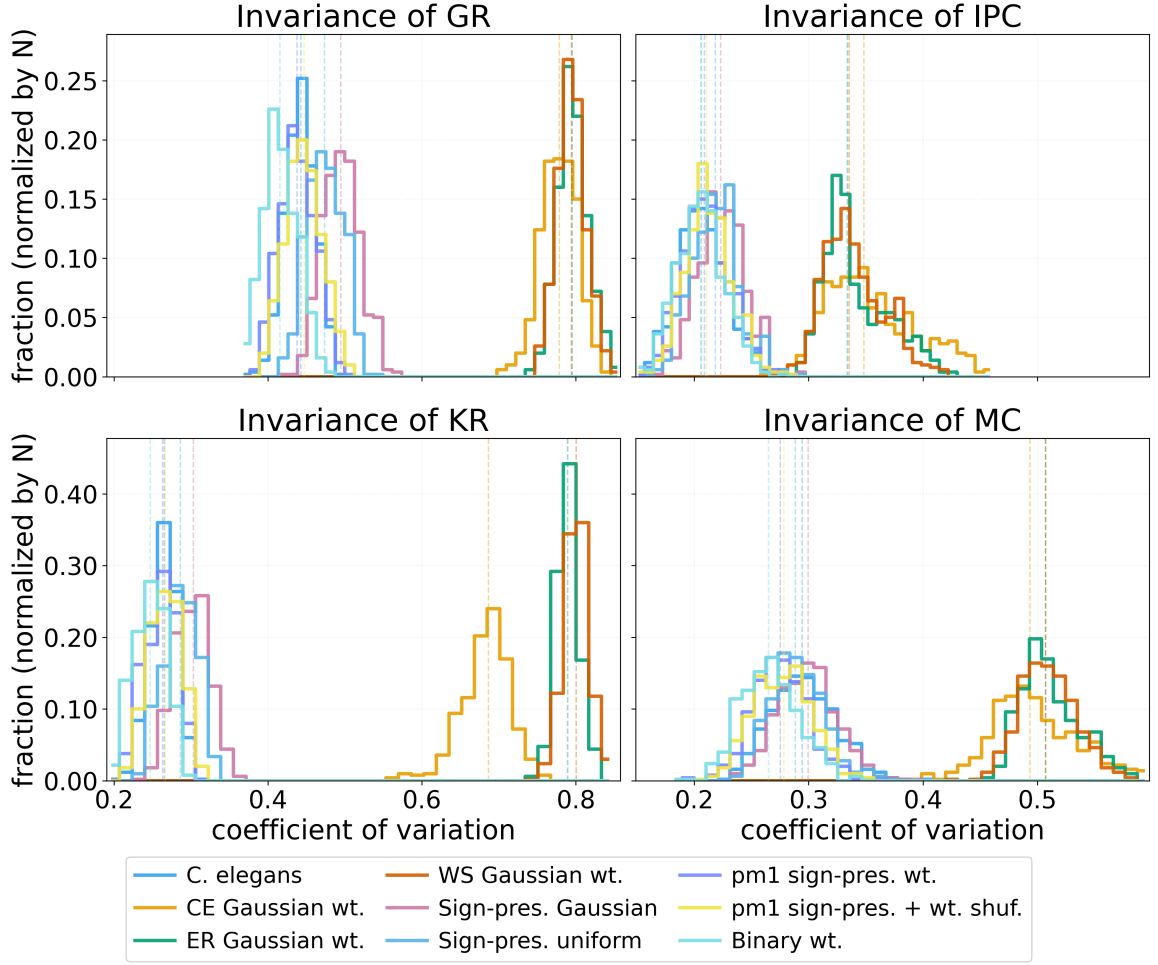


Figure 5.1: Invariance dispersion across network architectures, quantified as the coefficient of variation (CV) of each task-agnostic metric over hyperparameter sweeps. Panels show GR, IPC, KR, and MC. Curves are normalized histograms of CV (bin counts divided by the number of runs in that condition); dashed vertical lines mark medians. Lower CV indicates greater invariance to hyperparameter choice.

invariance. However, the three shuffled conditions are equivalent to one another for all four metrics. Weight shuffle, connection shuffle, and connection+weight shuffle therefore produce essentially the same dispersion profile once the original mapping between biological weights and edges has been disrupted. This suggests that, in the real-weight setting, much of the effect of connection shuffling is mediated through the reassignment of heterogeneous weights rather than through degree-preserving topology change by itself. The second experiment, shown in Fig. 5.3, Table 8.6, and Table 8.5, repeats the analysis after collapsing all nonzero weights to ± 1 while preserving the original sign assignment. In this setting, weight shuffle no longer changes magnitudes; it only redistributes positive and negative signs across the fixed topology. For GR, KR, and IPC, all four conditions are pairwise equivalent, indicating that once magnitude heterogeneity is removed these metrics are largely insensitive to both rewiring and sign redistribution. MC behaves differently. Four of the six MC comparisons satisfy equivalence, but the two topology-changing conditions are not equivalent to the unshuffled ± 1 baseline. Thus, after removing magnitude heterogeneity, MC still responds to rewiring, whereas GR, KR, and IPC do not.

The third experiment, shown in Fig. 5.4 and Table 8.7, isolates topology most directly by using binary weights. Here all nonzero edges have the same sign and magnitude, so weight shuffling is trivial and the only nontrivial perturbation is a degree-preserving connection shuffle. Under this manipulation, the binary base model and its shuffled counterpart are equivalent for IPC, KR, and GR. MC is absent from Table 8.7 because it does not satisfy the equivalence criterion; in Fig. 5.4, rewiring shifts MC to a lower-CV regime. In this binary setting, rewiring can no longer alter either magnitudes or signs, so the remaining MC effect provides direct evidence that topology itself can influence invariance.

Taken together, these three experiments show that the consequences of degree-preserving rewiring are strongly metric-dependent. For GR, KR, and IPC, most of the effect of rewiring in the real-weight network appears to come from disrupting the original placement of heterogeneous weights, because equivalence is largely restored once weights are collapsed to ± 1 or to a single binary value. MC is different. In the real-weight network, rewiring behaves much like weight shuffling, but in the ± 1 and binary settings MC remains sensitive to topology itself. The low CV of the empirical connectome therefore cannot be attributed to topology alone or to weight statistics alone; it reflects their interaction, with MC showing the clearest dependence on the detailed organization of the graph.

5.3 Half-normal weight perturbations have limited effect on invariance

We next isolate the effect of weight-magnitude distribution while holding connectivity and sign structure fixed. Starting from the *C. elegans* model, we perturb the magnitude of each nonzero weight while preserving its sign. For each nonzero entry W_{ij} ,

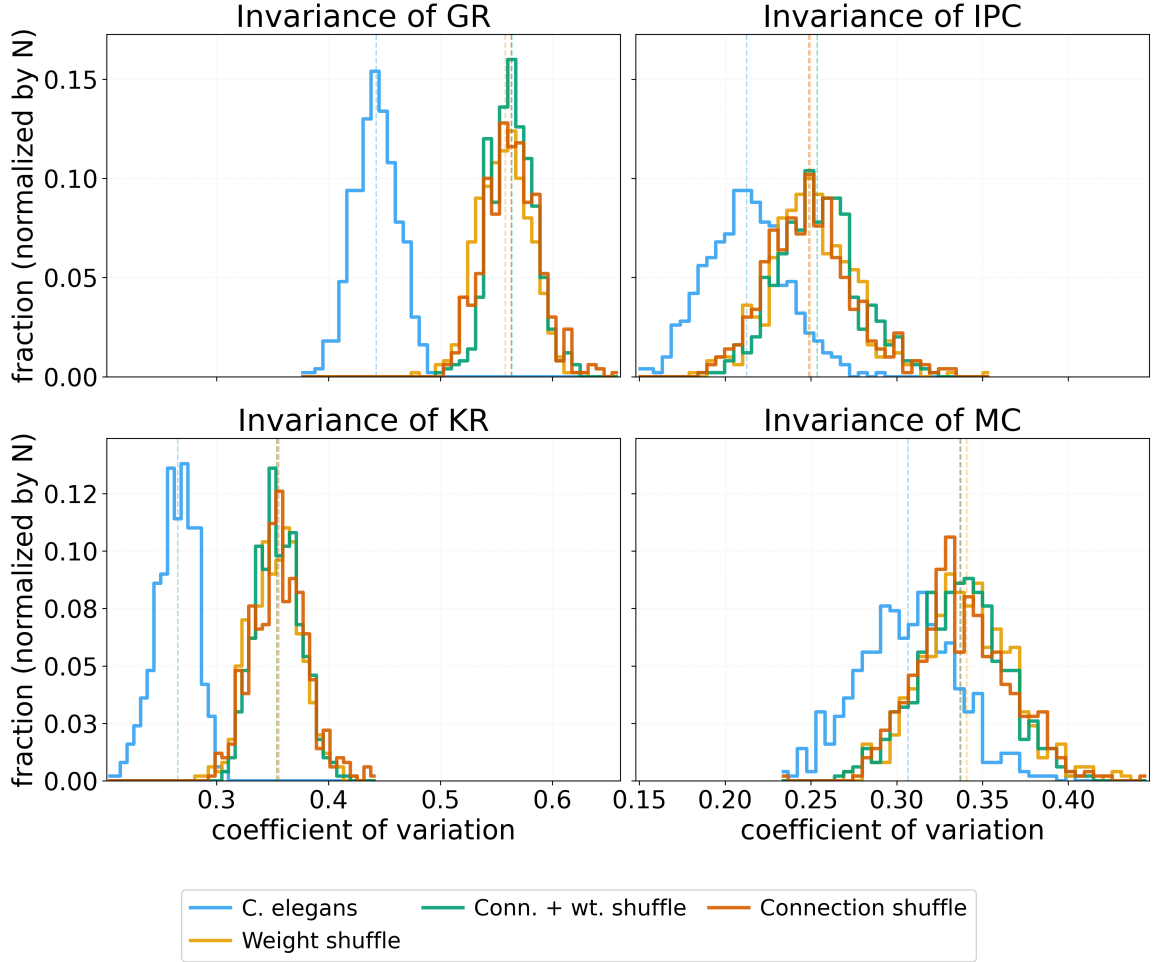


Figure 5.2: Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the original *C. elegans* weight matrix and its shuffled variants. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Blue: original *C. elegans* connectome; dark orange: weight shuffle; light orange: connection shuffle; red: connection+weight shuffle.

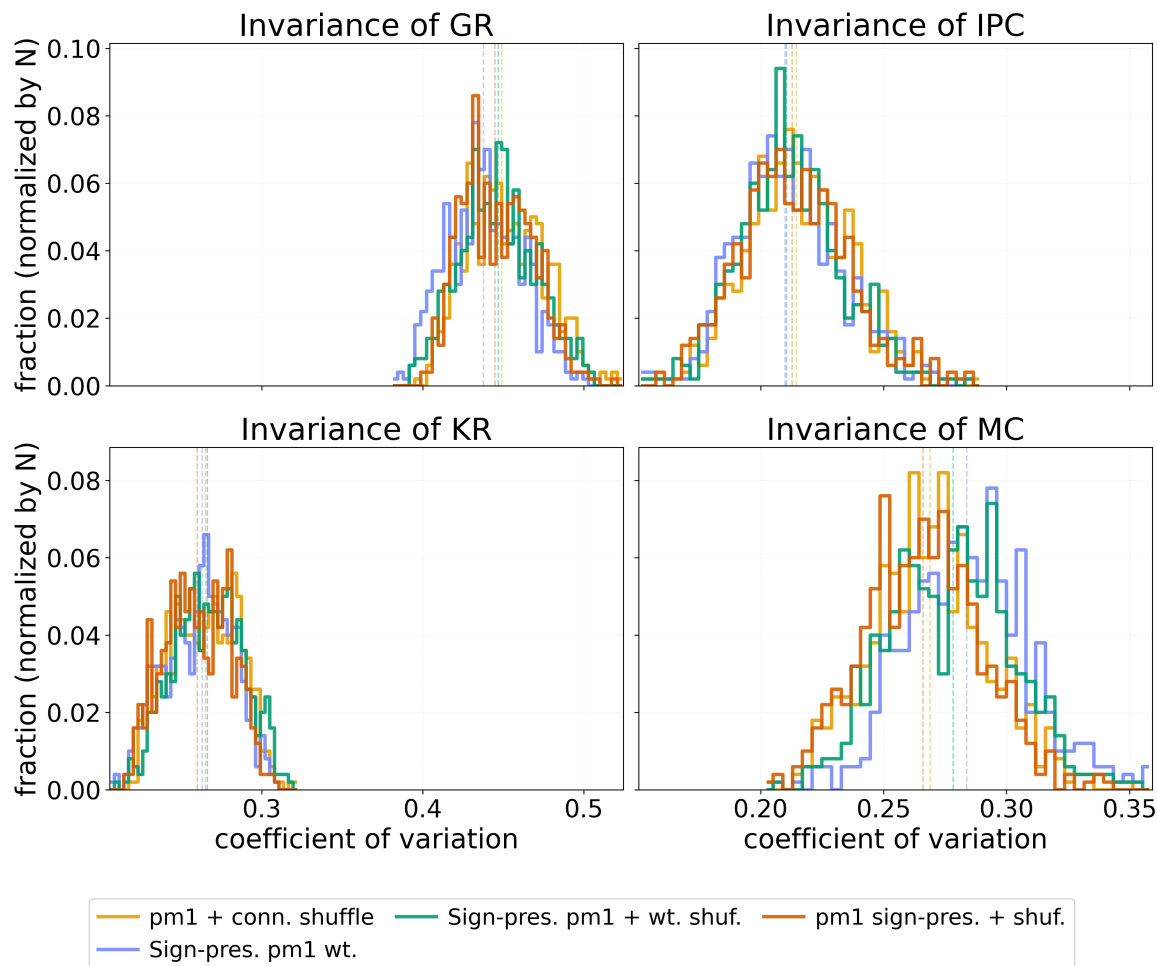


Figure 5.3: Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the *C. elegans* topology with ± 1 weights and its shuffled variants. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Blue: baseline *C. elegans* topology with ± 1 weights; gray: weight shuffle; pink: connection shuffle; purple: connection+weight shuffle.

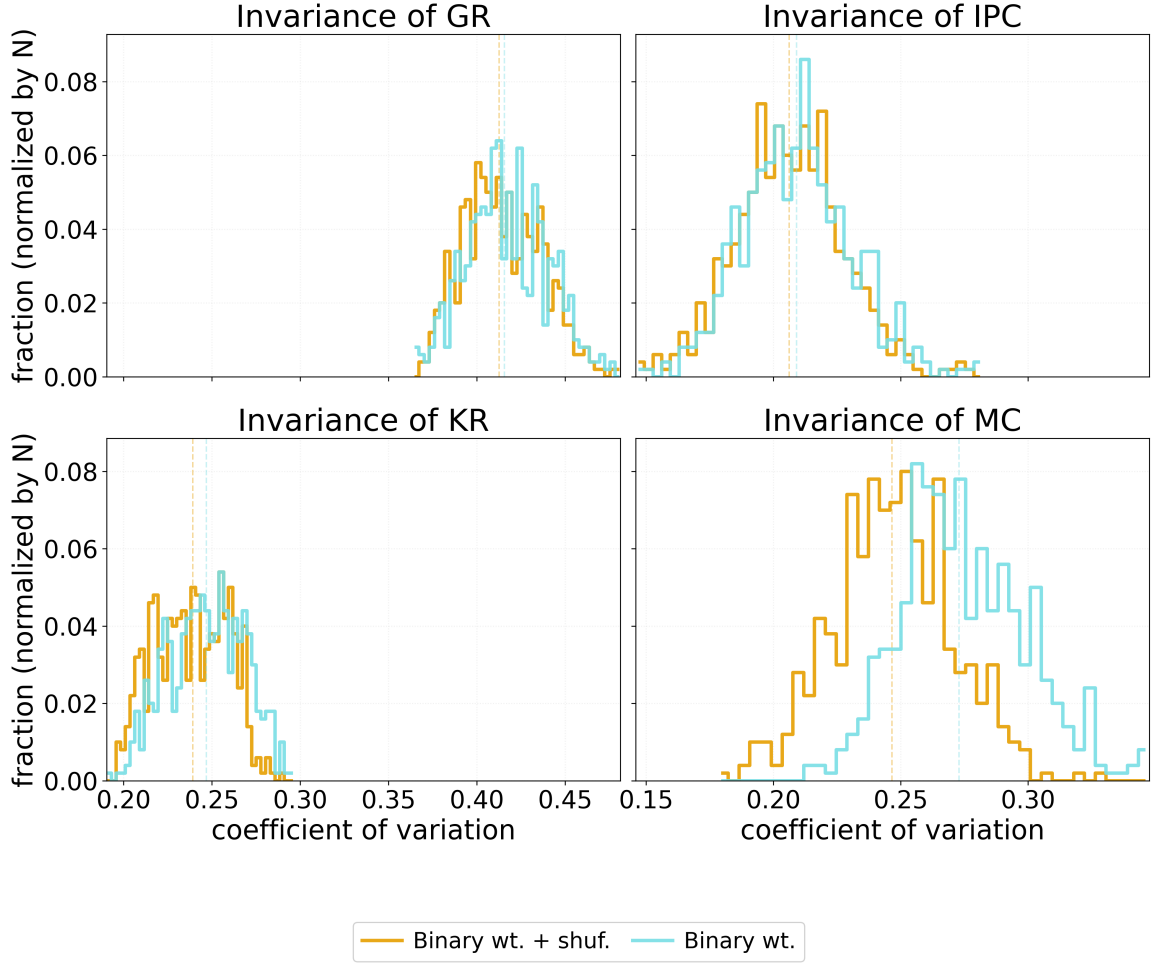


Figure 5.4: Distributions of invariance, measured as the coefficient of variation (CV) of each task-agnostic metric across hyperparameter sweeps, for the *C. elegans* topology with binary weights and its degree-preserving connection-shuffled counterpart. Panels correspond to GR, IPC, KR, and MC. Dashed vertical lines indicate medians. Lower CV indicates greater invariance to hyperparameter change. Turquoise: base-line binary-weight network; green: connection-shuffled binary-weight network.

we construct

$$W'_{ij} = \text{sign}(W_{ij}) |W_{ij} + \Lambda Z_{ij}|, \quad Z_{ij} \sim \mathcal{N}(0, 1).$$

where Λ controls the strength of the perturbation. As Λ increases, the empirical distribution of $|W|$ moves closer to a fitted shifted half-normal target, and its KL divergence from that target (Section 4.7) decreases. Because the edge set and sign pattern are unchanged, this experiment isolates the effect of the magnitude distribution from the effects of topology and global sign balance.

Preserving sign is necessary here because global sign balance strongly influences invariance (Section 5.4). If signs were allowed to change, any resulting change in mean performance or CV could reflect altered excitatory/inhibitory balance rather than the effect of the weight-magnitude distribution itself.

Figure 5.5 shows the relationship between KL divergence, mean performance, and hyperparameter sensitivity. As Λ increases, KL divergence decreases monotonically, but the resulting changes in both mean metric value and CV are modest. This is also reflected quantitatively in Tables 8.10 and 8.11: when sign is preserved, the average CV increases at $\Lambda = 10$ are 4.32% for MC, 8.76% for IPC, 11.6% for KR, and 10% for GR, while the corresponding mean metric changes are -2.48%, -0.499%, 9.67%, and 13.9%, respectively. MC and IPC change relatively little across the perturbation range, indicating that these metrics are largely insensitive to this transformation of weight magnitudes, while GR and KR vary somewhat more but still remain comparatively stable.

Compared with the version of the experiment in which sign is not preserved (Tables 8.8 and 8.9), the same perturbation produces much larger changes. At $\Lambda = 10$, CV rises by 45.6% for MC, 55.4% for IPC, 127% for KR, and 70.3% for GR, while the mean metric values increase by 75.3%, 73.4%, 251%, and 348%, respectively. This comparison supports the interpretation that the large effects seen under unconstrained perturbation are driven primarily by changes in sign balance rather than by the shift toward a half-normal magnitude distribution itself. For the *C. elegans* topology studied here, changing the weight-magnitude distribution alone has a limited effect on hyperparameter invariance or performance, especially for MC and IPC.

5.4 Global neurotransmitter balance shapes the performance–invariance tradeoff

To isolate the effect of neurotransmitter balance, we performed a controlled sign-flipping experiment in which a fraction f of randomly selected synapses had their signs flipped while preserving connectivity and weight magnitudes (Figures 5.6, 5.7, 5.8). This procedure continuously perturbs the global excitatory/inhibitory balance while holding all other structural properties fixed.

In Figures 5.6 and 5.7, the endpoint with negative sign fraction = 0 corresponds

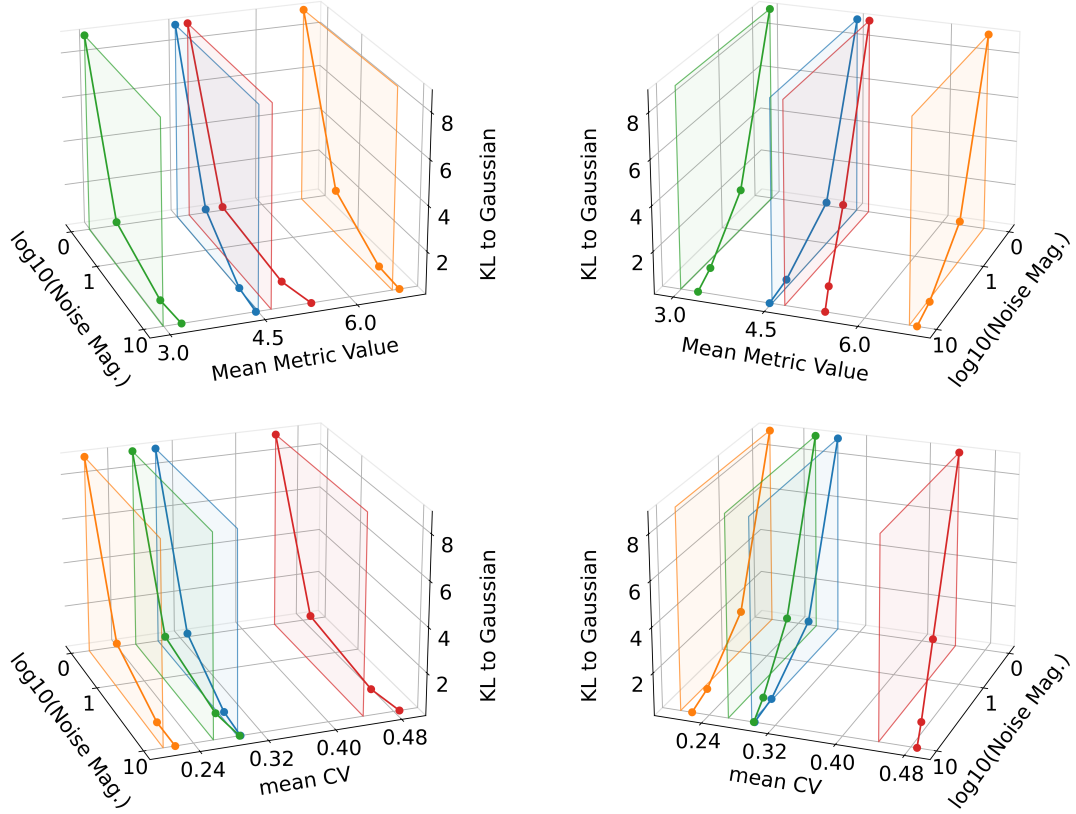


Figure 5.5: Effect of perturbing nonzero weight magnitudes toward a half-normal distribution while preserving connectivity and edge sign. Starting from the *C. elegans* network, nonzero weights are transformed as $W'_{ij} = \text{sign}(W_{ij}) |W_{ij} + \Lambda Z_{ij}|$, $Z_{ij} \sim \mathcal{N}(0, 1)$, where Λ controls perturbation strength. The x-axis shows the KL divergence between the empirical distribution of $|W|$ and a fitted shifted half-normal target on a $\log_{10}$ scale; increasing Λ decreases this divergence. Top row: mean metric value versus KL divergence. Bottom row: mean CV versus KL divergence. Colored trajectories correspond to GR (green), KR (red), IPC (orange), and MC (blue). Left and right panels show the same data from two viewing angles. The transparent planes indicate the unperturbed baseline, corresponding to $\Lambda = 0$.

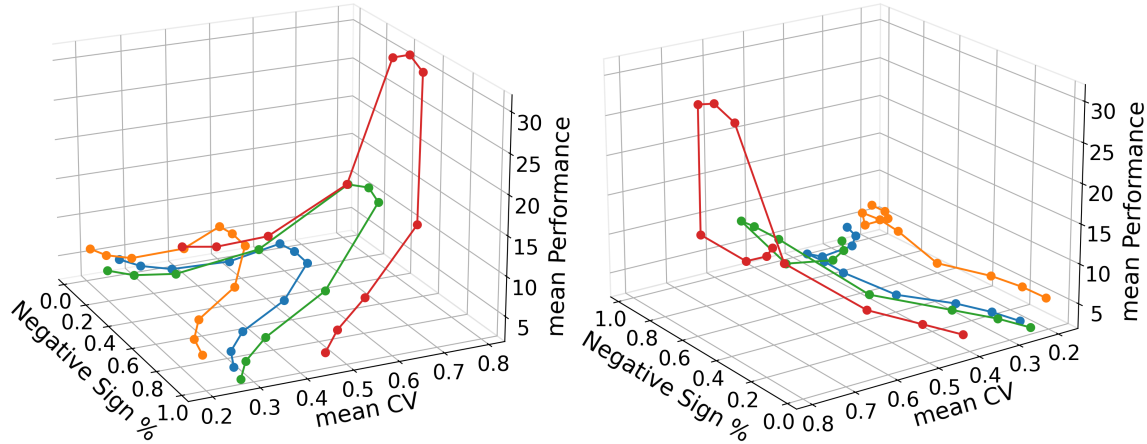


Figure 5.6: IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the original connectome using $\pm$ weights. Increasing sign flip percentage shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. A total of 250 trials were performed for each fraction.

to

$$W'_{ij} = \begin{cases} 1 & \text{if } W_{ij} \neq 0 \\ 0 & \text{if } W_{ij} = 0 \end{cases}$$

In this binary-positive limit, both networks exhibit maximal invariance but reduced mean computational performance. As the fraction of flipped synapses increases, both performance and CV generally increase. Mean computational performance is typically highest near a balanced excitatory/inhibitory regime, but this gain is accompanied by substantially increased sensitivity to hyperparameter variation. The dominant pattern is therefore a tradeoff: moving toward global sign balance increases mean performance at the cost of task-agnostic invariance. This relationship is not perfectly monotonic in every metric, but the overall trend is consistent across the experiments shown here.

Figure 5.8 also shows where the original *C. elegans* network lies on this CV–performance curve. Within the reservoir abstraction used here (static weights, tanh nonlinearity, spectral-radius normalization, contact-count proxy weights, and a coarse neurotransmitter-to-sign mapping), the *C. elegans* instantiation occupies a relatively low-variance regime but does not maximize mean task-agnostic performance. Notably, the original connectome also reaches a substantially lower performance peak than the topology-matched ± 1 network.

Taken together, these results indicate that global neurotransmitter balance is a major determinant of the performance invariance tradeoff. Networks with highly asymmetric sign structure exhibit reduced computational capacity but greater invariance, whereas networks approaching a balanced excitatory/inhibitory regime achieve higher performance at the cost of increased variability under hyperparameter change.

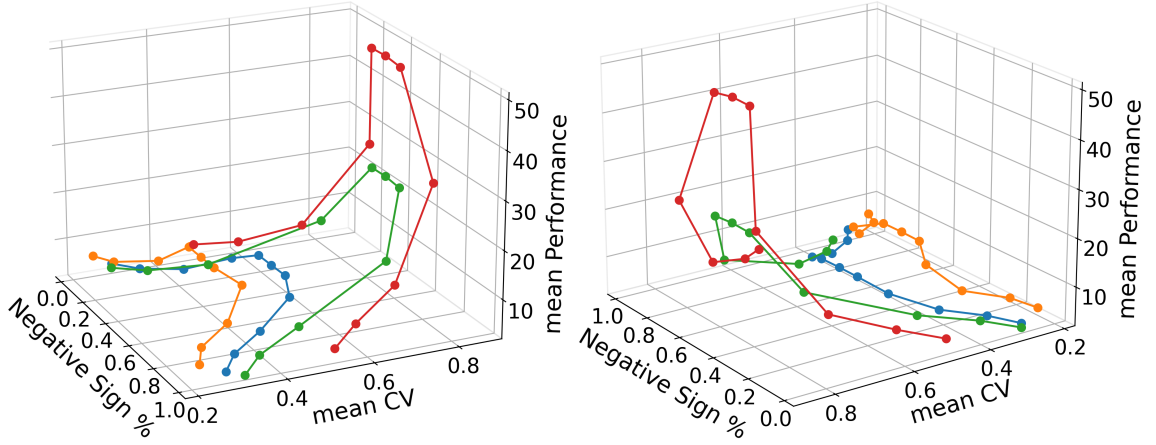


Figure 5.7: IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the ER random graph using binary weights. Increasing sign flip percentage shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. A total of 250 trials were performed for each fraction.

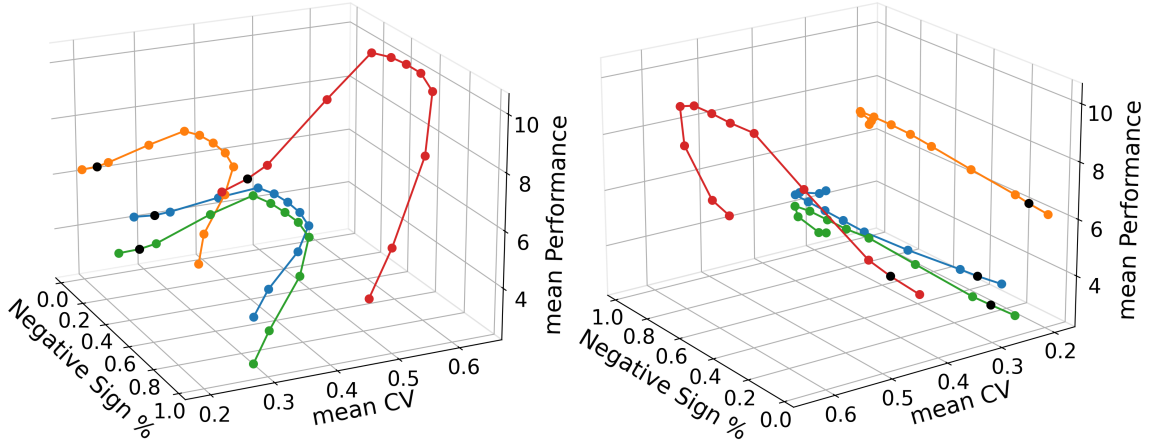


Figure 5.8: IPC is denoted in orange, GR in green, MC in blue, and KR in red. Mean computational coefficient of variation and mean performance are shown as a function of the fraction of synapse signs flipped from the absolute value of the *C. elegans* connectome. The black point represents the negative sign percentage observed in the original connectome. A total of 250 trials were performed for each fraction.

Chapter 6

Discussion

Our results place the *C. elegans* connectome in a comparatively low-variance regime across multiple task-agnostic reservoir metrics, but the updated ablations show that this regime is not explained by any single coarse architectural property. Across the architecture-level controls, preserving global excitatory/inhibitory balance is sufficient to recover much of the connectome’s invariance profile for IPC, KR, and GR, even when exact sign placement or empirical weight magnitudes are simplified. MC behaves differently: it is not recovered by the sign-preserving ± 1 controls and is matched only by architectures that retain heterogeneous weights while preserving topology and sign. The overall picture is therefore layered, with different metrics depending on different combinations of sign balance, weight heterogeneity, and topology.

The shuffling experiments clarify how topology enters this picture. In the real-weight network, degree-preserving connection shuffling produces nearly the same dispersion profile as weight shuffling, and the combined shuffle is equivalent to both. This suggests that much of the apparent effect of rewiring in the empirical network comes from disrupting the assignment of heterogeneous biological weights to edges, rather than from topology change alone. Once magnitude heterogeneity is removed, GR, KR, and IPC become largely insensitive to rewiring, whereas MC remains sensitive in both the ± 1 and binary controls. Within the reservoir abstraction used here, topology therefore has its clearest effect on MC, while for the other metrics much of the rewiring effect can be explained by changes in weight placement.

The weight-distribution experiments refine this interpretation. At the architecture level, replacing the empirical magnitude distribution with sign-preserving Gaussian or uniform samples changes the equivalence pattern relative to the original connectome, especially for KR, IPC, and GR. In contrast, continuously perturbing the magnitude distribution toward a half-normal target while preserving topology and sign produces only modest changes in CV and mean performance. Taken together, these results suggest that large changes in invariance arise primarily when sign balance is altered or, for MC, when heterogeneous weights are reassigned across edges, whereas deforming the magnitude distribution on a fixed signed topology has a much smaller effect.

The sign-flipping experiments place these structural results in a broader performance-invariance context. Moving toward a more balanced excitatory/inhibitory regime generally increases mean task-agnostic performance but also increases CV, whereas

strongly asymmetric sign structure yields lower-performance but more invariant reservoirs. Under the present modeling choices, the empirical *C. elegans* instantiation lies on the lower-variance side of this tradeoff rather than at the peak-performance region. This does not show that the biological network is optimized for invariance, but suggests that stable computation under parameter variation is a constraint shaping the excitatory/inhibitory organization of biological circuits.

That interpretation should be treated cautiously because the connectome model compresses substantial biological uncertainty into a static signed network. In our instantiation, approximately 6% of connections are inhibitory, but recent sign-inference work suggests the inhibitory fraction may be closer to 20% (Fenyves et al., 2020). If so, the empirical connectome may lie further along the observed performance–CV curve than our current model suggests, with correspondingly higher performance and CV. In addition, excitatory/inhibitory balance is not fixed across the life of the organism: developmental and aging-related changes in inhibitory signaling imply that the effective location of the *C. elegans* nervous system on this tradeoff may vary across the lifespan (Tanis et al., 2009; Wirak et al., 2022). Direct biological interpretation is also limited by our coarse neurotransmitter-to-sign mapping, by the treatment of non-GABA synapses as excitatory, and by the fact that inputs are delivered to all neurons without biologically segregated input and output populations. The present model should therefore be understood as a structural reservoir abstraction rather than a faithful dynamical simulation of the animal.

Taken together, these results suggest that computational invariance in connectome reservoirs is distributed across multiple structural scales. Global sign balance, which appears to be the dominant control parameter in these experiments, explains much of the low-variance behavior of IPC, KR, and GR, whereas heterogeneous weight organization appears especially important for MC. Topology has its clearest independent effect on MC once the effects of weight heterogeneity and, in the binary control, sign variation have been removed. For reservoir computing, this implies that sign balance is especially important, while topology and weight heterogeneity may be specifically useful for improving MC performance. For neuroscience, the results suggest that connectome structure may support stable computation through the coordinated interaction of sign organization, weight heterogeneity, and recurrent wiring, with global sign balance emerging here as a particularly strong determinant of computational invariance.

6.1 Future directions

Several extensions would help clarify these interpretations. First, the model comparisons could be strengthened by adding additional topology controls that match degree and sign statistics more tightly, including degree-matched random and scale-free networks. Such controls would help determine which residual effects are genuinely topological and whether the distinctive sensitivity of MC persists outside the *C. elegans* degree distribution.

Second, it would be useful to vary weight heterogeneity more directly. In particu-

lar, interpolating between a binary-weight sign-preserving model and a heterogeneous sign-preserving model would help clarify the extent to which MC depends on the presence of heterogeneous magnitudes themselves, as opposed to the specific arrangement of those magnitudes on the graph.

Third, the present conclusions should be tested on downstream tasks such as NARMA to determine whether the same structural relationships hold when reservoirs are evaluated by task performance rather than task-agnostic statistics alone. The same framework could also be used to investigate how connectome optimization algorithms (Churchland & Garcia-Ojalvo, 2026) alter task-agnostic statistics, and whether task-driven optimization moves biological reservoirs along the same performance–invariance axes identified here.

Fourth, it would be valuable to repeat these experiments with biologically segregated input and output populations. Using sensory neurons as inputs and muscles or motor neurons as outputs would bring the model closer to a functional circuit interpretation and would better connect this work to prior studies of connectome-based computation (Kawai et al., 2019). Finally, the roles of sign inference and empirical weight placement deserve more direct study. Natural next steps would be to compare alternative neurotransmitter-to-sign mappings and to interpolate between the empirical weight assignment and progressively shuffled assignments while tracking both CV and performance.

Chapter 7

AI Usage Disclosure

Artificial intelligence tools, specifically ChatGPT (OpenAI, GPT-5 series) and Codex (OpenAI), were used as assistive tools during the course of this work.

AI tools were used to review text for clarity, grammar, and structure, and to identify ambiguous or overstated claims. All text included in this manuscript was written, reviewed, and edited by the authors, and any AI-assisted edits were manually verified and revised as necessary to ensure accuracy.

AI tools were also used to assist in identifying relevant literature. These tools were used in a search-assistance capacity to locate papers related to reservoir computing and recurrent biological networks. All cited papers were independently located, read, and verified by the author prior to inclusion.

In addition, AI tools were used as a programming reference to identify relevant libraries, functions, and debugging strategies. They were also used to assist in generating example plotting code and suggesting implementation approaches. All code used in this project was written, reviewed, and validated by the author.

AI tools were used solely as assistive instruments. All scientific claims, analysis, interpretations, and conclusions presented in this work are the independent work of the author.

Chapter 8

Supplemental information

8.1 Neuroscience Terms Explained for Computer Scientists

- **neuron** – basic building block of neural circuits. In this work, neurons are represented as nodes on a graph, each with both an activation function and an internal state representing membrane potential.
- **synapse** – another basic building block of neural circuits. In this work, synapses are represented by directed edges with weights connecting two neurons.
- **excitatory neuron** – a neuron with only positive-weight outgoing edges
- **inhibitory neuron** – a neuron with only negative-weight outgoing edges
- **E/I balance** – the ratio of excitatory neurons to inhibitory neurons
- **neurotransmitter** – a chemical that allows neurons to communicate with each other by transmitting signals across synapses (Sheffler et al., 2025).
- **gap junction** – a type of synapse that connects the cytoplasm of two adjacent cells; bidirectional with the same weight for both sides. In our modeling, these are either removed or treated as two distinct positively signed edges.
- **neuromuscular junction** – a specialized synapse where a motor neuron communicates with a muscle fiber. These directed edges point to non-neural cells and hence are not included in our graph.
- **connectome** – a wiring diagram of a biological neural network. This typically contains all neurons and synapses, along with edge weights.
- **sensory neuron** – a neuron that receives sensory input. During typical biological use, these can be thought of as “inputs” to the network.
- **motor neuron** – a neuron that passes signals to motor cells. During typical biological use, these can be thought of as the “outputs” of the network.

- **synaptic terminals** – bulbous endings of a neuron’s axon where neurotransmitters are transferred to other cells (neurons or muscles). We use the number of these terminals as a proxy for edge weight.
- **membrane potential** – the difference in electrical charge between the inside and outside of a neuron, forming the basis for neural activity (Henley, 2021). For our purposes, this is the internal state of the neuron representing how much our activation function will be saturated.
- **small-worldness** – a property commonly observed in biological networks. A graph that exhibits the small-world property has both a high clustering coefficient and a short average path length. This structure is claimed to make information spread more efficient and to make the network more robust (Bassett & Bullmore, 2007; Latora & Marchiori, 2001).
- **scale-free network** – a network whose degree distribution follows a power law, meaning the fraction of nodes with degree k follows a power law $k^{-\alpha}$ (Broido & Clauset, 2019).

8.2 Equivalence testing

Equivalence testing using TOST (Table 8.1 and Table 8.2) further clarifies these relationships. We used TOST equivalence testing on the mean dispersion (CV) between each architecture and the biological connectome. The equivalence margin was set to $\Delta = |0.05(\text{median}(\text{dispersion}))|$, computed per metric across all runs. We concluded equivalence when both one-sided tests rejected the null that the mean difference lay outside $[-\Delta, +\Delta]$. Thus, small p-values indicate that the dispersion difference is statistically confined within the predefined practical tolerance. Under this criterion, only specific sign-preserving perturbations satisfied equivalence for selected metrics, while most architectures—including connection shuffles, weight shuffles, and random graph controls—did not satisfy equivalence across all metrics simultaneously. Failure to conclude equivalence (large p) does not imply non-equivalence; it indicates insufficient evidence that the mean difference is within $\pm\Delta$ under the chosen margin and sample size. Each comparison used 1000 observations.

8.3 Tables

Table 8.1: Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity.

Metric	Comparison	TOST bound	p
MC	Binary wt. vs pm1 sign-pres. + wt. shuf.	0.0151	0.007238
MC	Binary wt. vs pm1 sign-pres. wt.	0.0151	5.5e-05
MC	CE Gaussian wt. vs WS Gaussian wt.	0.0151	0.02401
MC	ER Gaussian wt. vs WS Gaussian wt.	0.0151	5.706e-16
MC	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.0151	3.329e-16
MC	pm1 sign-pres. + wt. shuf. vs Sign-pres. uniform	0.0151	0.04148
MC	Sign-pres. Gaussian vs Sign-pres. uniform	0.0151	0.0001002
MC	Sign-pres. Gaussian vs C. elegans	0.0151	9.184e-12
MC	Sign-pres. uniform vs C. elegans	0.0151	9.91e-09
IPC	Binary wt. vs pm1 sign-pres. + wt. shuf.	0.01138	6.972e-05
IPC	Binary wt. vs pm1 sign-pres. wt.	0.01138	2.129e-08
IPC	Binary wt. vs C. elegans	0.01138	5.563e-14
IPC	ER Gaussian wt. vs WS Gaussian wt.	0.01138	8.612e-08
IPC	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.01138	2.079e-11
IPC	pm1 sign-pres. + wt. shuf. vs Sign-pres. uniform	0.01138	1.807e-05
IPC	pm1 sign-pres. + wt. shuf. vs C. elegans	0.01138	1.779e-06
IPC	Sign-pres. Gaussian vs Sign-pres. uniform	0.01138	5.499e-06
IPC	pm1 sign-pres. wt. vs Sign-pres. uniform	0.01138	0.01166
IPC	pm1 sign-pres. wt. vs C. elegans	0.01138	1.938e-10

Table 8.2: Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank.

Metric	Comparison	TOST bound	p
KR	ER Gaussian wt. vs WS Gaussian wt.	0.01441	5.98e-05
KR	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.01441	1.116e-11
KR	pm1 sign-pres. + wt. shuf. vs C. elegans	0.01441	3.929e-21
KR	pm1 sign-pres. wt. vs C. elegans	0.01441	2.716e-20
GR	Binary wt. vs pm1 sign-pres. wt.	0.02375	0.0459
GR	CE Gaussian wt. vs ER Gaussian wt.	0.02375	3.902e-06
GR	CE Gaussian wt. vs WS Gaussian wt.	0.02375	3.849e-08
GR	ER Gaussian wt. vs WS Gaussian wt.	0.02375	5.565e-65
GR	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.02375	1.613e-24
GR	pm1 sign-pres. + wt. shuf. vs C. elegans	0.02375	4.416e-41
GR	Sign-pres. Gaussian vs Sign-pres. uniform	0.02375	0.03561
GR	pm1 sign-pres. wt. vs C. elegans	0.02375	9.945e-45

Table 8.3: Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity for shuffling experiment 1.

Metric	Comparison	TOST bound	p
MC	Conn. + wt. shuf. vs Conn. shuf.	0.01656	7.926e-20
MC	Conn. + wt. shuf. vs Weight shuf.	0.01656	1.844e-14
MC	Conn. shuf. vs Weight shuf.	0.01656	1.615e-15
IPC	Conn. + wt. shuf. vs Conn. shuf.	0.01213	6.19e-07
IPC	Conn. + wt. shuf. vs Weight shuf.	0.01213	4.58e-09
IPC	Conn. shuf. vs Weight shuf.	0.01213	2.501e-12

Table 8.4: Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 1.

Metric	Comparison	TOST bound	p
KR	Conn. + wt. shuf. vs Conn. shuf.	0.01722	4.406e-30
KR	Conn. + wt. shuf. vs Weight shuf.	0.01722	2.781e-27
KR	Conn. shuf. vs Weight shuf.	0.01722	8.019e-21
GR	Conn. + wt. shuf. vs Conn. shuf.	0.02759	4.394e-70
GR	Conn. + wt. shuf. vs Weight shuf.	0.02759	7.459e-46
GR	Conn. shuf. vs Weight shuf.	0.02759	4.201e-41

Table 8.5: Pairwise equivalence tests (TOST) across modes within information processing capacity and memory capacity for shuffling experiment 2.

Metric	Comparison	TOST bound	p
MC	pm1 sign-pres. + shuf. vs pm1 + conn. shuf.	0.01368	7.998e-12
MC	pm1 sign-pres. + shuf. vs pm1 sign-pres. + wt. shuf.	0.01368	0.04546
MC	pm1 + conn. shuf. vs pm1 sign-pres. + wt. shuf.	0.01368	7.225e-05
MC	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.01368	2.114e-05
IPC	pm1 sign-pres. + shuf. vs pm1 + conn. shuf.	0.0106	8.308e-10
IPC	pm1 sign-pres. + shuf. vs pm1 sign-pres. + wt. shuf.	0.0106	2.293e-11
IPC	pm1 sign-pres. + shuf. vs pm1 sign-pres. wt.	0.0106	7.678e-11
IPC	pm1 + conn. shuf. vs pm1 sign-pres. + wt. shuf.	0.0106	1.491e-07
IPC	pm1 + conn. shuf. vs pm1 sign-pres. wt.	0.0106	3.382e-07
IPC	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.0106	7.407e-15

Table 8.6: Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 2.

Metric	Comparison	TOST bound	p
KR	pm1 sign-pres. + shuf. vs pm1 + conn. shuf.	0.01317	4.991e-12
KR	pm1 sign-pres. + shuf. vs pm1 sign-pres. + wt. shuf.	0.01317	6.44e-08
KR	pm1 sign-pres. + shuf. vs pm1 sign-pres. wt.	0.01317	2.063e-20
KR	pm1 + conn. shuf. vs pm1 sign-pres. + wt. shuf.	0.01317	4.512e-16
KR	pm1 + conn. shuf. vs pm1 sign-pres. wt.	0.01317	1.584e-13
KR	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.01317	3.403e-09
GR	pm1 sign-pres. + shuf. vs pm1 + conn. shuf.	0.02224	3.981e-30
GR	pm1 sign-pres. + shuf. vs pm1 sign-pres. + wt. shuf.	0.02224	1.419e-47
GR	pm1 sign-pres. + shuf. vs pm1 sign-pres. wt.	0.02224	9.588e-23
GR	pm1 + conn. shuf. vs pm1 sign-pres. + wt. shuf.	0.02224	7.492e-30
GR	pm1 + conn. shuf. vs pm1 sign-pres. wt.	0.02224	9.344e-10
GR	pm1 sign-pres. + wt. shuf. vs pm1 sign-pres. wt.	0.02224	1.875e-20

Table 8.7: Pairwise equivalence tests (TOST) across modes within kernel rank and generalization rank for shuffling experiment 3.

Metric	Comparison	TOST bound	p
IPC	Binary wt. vs Binary wt. + shuf.	0.01039	2.887e-08
KR	Binary wt. vs Binary wt. + shuf.	0.01215	0.002745
GR	Binary wt. vs Binary wt. + shuf.	0.02068	5.831e-35

Table 8.8: Changes in mean CV across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is not preserved in this version of the experiment.

Metric	Baseline	$\Lambda = 1$	$\Lambda = 5$	$\Lambda = 10$
MC	0.3548	+0.00808 (+2.28%)	+0.0851 (+24%)	+0.162 (+45.6%)
IPC	0.2718	+0.00868 (+3.19%)	+0.0891 (+32.8%)	+0.151 (+55.4%)
KR	0.2634	+0.00505 (+1.92%)	+0.125 (+47.6%)	+0.335 (+127%)
GR	0.4426	+0.00829 (+1.87%)	+0.162 (+36.5%)	+0.311 (+70.3%)

Table 8.9: Changes in mean metric value across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is not preserved in this version of the experiment.

Metric	Baseline	$\Lambda = 1$	$\Lambda = 5$	$\Lambda = 10$
MC	4.161	+0.00525 (+0.126%)	+0.715 (+17.2%)	+3.13 (+75.3%)
IPC	5.863	+0.0241 (+0.411%)	+0.999 (+17%)	+4.31 (+73.4%)
KR	2.99	+0.0545 (+1.82%)	+1.6 (+53.4%)	+7.51 (+251%)
GR	4.712	+0.12 (+2.55%)	+3.54 (+75%)	+16.4 (+348%)

Table 8.10: Changes in mean CV across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is preserved in this version of the experiment.

Metric	Baseline	$\Lambda = 1$	$\Lambda = 5$	$\Lambda = 10$
MC	0.3623	+0.00232 (+0.641%)	+0.0133 (+3.67%)	+0.0156 (+4.32%)
IPC	0.2781	+0.00298 (+1.07%)	+0.0178 (+6.38%)	+0.0244 (+8.76%)
KR	0.2634	+0.00294 (+1.12%)	+0.0211 (+8.01%)	+0.0306 (+11.6%)
GR	0.4426	+0.00508 (+1.15%)	+0.0319 (+7.2%)	+0.0444 (+10%)

Table 8.11: Changes in mean metric value across Gaussian noise magnitudes. Baseline column lists the reference value for each metric; other entries show absolute change followed by percent change. Relative to baseline mode weight_test0.0. Sign is preserved in this version of the experiment.

Metric	Baseline	$\Lambda = 1$	$\Lambda = 5$	$\Lambda = 10$
MC	4.096	-0.00846 (-0.206%)	-0.094 (-2.3%)	-0.102 (-2.48%)
IPC	5.733	+0.00524 (+0.0914%)	-0.0257 (-0.448%)	-0.0286 (-0.499%)
KR	2.99	+0.0314 (+1.05%)	+0.193 (+6.45%)	+0.289 (+9.67%)
GR	4.712	+0.0731 (+1.55%)	+0.447 (+9.49%)	+0.655 (+13.9%)

Bibliography

- Barabási, A.-L. (2016). *Network science*. Cambridge University Press. <https://www.cambridge.org/fr/academic/subjects/physics/statistical-physics/network-science?format=HB&isbn=9781107076266>
- Bassett, D., & Bullmore, E. (2007). Small-world brain networks. *The Neuroscientist : a review journal bringing neurobiology, neurology and psychiatry*, 12, 512–23. <https://doi.org/10.1177/1073858406293182>
- Broido, A. D., & Clauset, A. (2019). Scale-free networks are rare. *Nature Communications*, 10(1). <https://doi.org/https://doi.org/10.1038/s41467-019-08746-5>
- Burden, S. A., Libby, T., Jayaram, K., Sponberg, S., & Donelan, J. M. (2024). Why animals can outrun robots. *Science Robotics*, 9(89). <https://doi.org/https://doi.org/10.1126/scirobotics.adi9754>
- Casal, M. A., Galella, S., Vilarroya, O., & Garcia-Ojalvo, J. (2020). Soft-wired long-term memory in a natural recurrent neuronal network. *Chaos*, 30(6), 061101. <https://doi.org/10.1063/5.0009709>
- Churchland, M. W., & Garcia-Ojalvo, J. (2026). Reinforcement learning in densely recurrent biological networks. *iScience*, 29(1), 114436. <https://doi.org/https://doi.org/10.1016/j.isci.2025.114436>
- Dambre, J., Verstraeten, D., Schrauwen, B., & Massar, S. (2012). Information processing capacity of dynamical systems. *Scientific Reports*, 2, 514. <https://doi.org/10.1038/srep00514>
- Fenyves, B. G., Szilágyi, G. S., Vassy, Z., Sóti, C., & Csermely, P. (2020). Synaptic polarity and sign-balance prediction using gene expression data in the caenorhabditis elegans chemical synapse neuronal connectome network. *PLoS Computational Biology*, 16(12), e1007974–e1007974. <https://doi.org/https://doi.org/10.1371/journal.pcbi.1007974>
- Fernando, C., & Sojakka, S. (2003). Pattern recognition in a bucket, 588–597. https://doi.org/https://doi.org/10.1007/978-3-540-39432-7_63
- Fukushima, K. (1980). Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4), 193–202. <https://doi.org/https://doi.org/10.1007/bf00344251>
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665–673. <https://doi.org/https://doi.org/10.1038/s42256-020-00257-z>

- Gleeson, P., Lung, D., Grosu, R., Hasani, R., & Larson, S. D. (2018). C302: A multiscale framework for modelling the nervous system of *Caenorhabditis elegans*. *Philosophical Transactions of the Royal Society B*, 373(1758), 20170379. <https://doi.org/10.1098/rstb.2017.0379>
- Henley, C. (2021). *Foundations of neuroscience casey henley*. Michigan State University.
- Humphries, M. D., & Gurney, K. (2008). Network “small-world-ness”: A quantitative method for determining canonical network equivalence (O. Sporns, Ed.). *PLoS ONE*, 3(4), e0002051. <https://doi.org/https://doi.org/10.1371/journal.pone.0002051>
- Inoue, K., Kubota, T., Hoan Tran, Q., Akashi, N., Terajima, R., Kabayama, T., Guan, J., & Nakajima, K. (2026). Reservoir computing bootcamp—from python/numpy tutorial for the complete beginners to cutting-edge research topics of reservoir computing. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 36(2), 023109. <https://doi.org/10.1063/5.0283386>
- Jaeger, H. (2001). *The “echo state” approach to analysing and training recurrent neural networks* (tech. rep. No. 148) (Corrected version (2010) available at <https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf>). GMD – German National Research Center for Information Technology.
- Jaeger, H. (2002). *Short term memory in echo state networks* (GMD-Report No. 152). GMD – German National Research Institute for Computer Science. <https://www.ai.rug.nl/minds/uploads/STMEchoStatesTechRep.pdf>
- Jaeger, H., & Haas, H. (2004). Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667), 78–80. <https://doi.org/10.1126/science.1091277>
- Kawai, Y., Park, J., & Asada, M. (2019). A small-world topology enhances the echo state property and signal propagation in reservoir computing. *Neural Networks*, 112, 15–23. <https://doi.org/https://doi.org/10.1016/j.neunet.2019.01.002>
- Latora, V., & Marchiori, M. (2001). Efficient behavior of small-world networks. *Physical review letters*, 87, 198701. <https://doi.org/10.1103/PhysRevLett.87.198701>
- Lindsay, G. (2020). Convolutional neural networks as a model of the visual system: Past, present, and future. *Journal of Cognitive Neuroscience*, 33(10), 1–15. https://doi.org/https://doi.org/10.1162/jocn_a_01544
- Lukoševičius, M. (2012). A practical guide to applying echo state networks. In G. Montavon, G. B. Orr, & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 659–686, Vol. 7700). Springer. https://doi.org/10.1007/978-3-642-35289-8_36
- Lukoševičius, M., Jaeger, H., & Schrauwen, B. (2012). Reservoir computing trends. *KI - Künstliche Intelligenz*, 26(4), 365–371. <https://doi.org/10.1007/s13218-012-0204-5>
- Maass, W., Legenstein, R., & Bertschinger, N. (2004). Methods for estimating the computational power and generalization capability of neural microcircuits. *Advances in Neural Information Processing Systems 17 (NIPS 2004)*, 865–872.

- Nilsson, N. J. (2010, October). *The quest for artificial intelligence*. Cambridge University Press.
- Pearson, K. G. (2000). Neural adaptation in the generation of rhythmic behavior. *Annual Review of Physiology*, 62(1), 723–753. <https://doi.org/https://doi.org/10.1146/annurev.physiol.62.1.723>
- Pieters, O., De Swaef, T., Stock, M., & wyffels, F. (2022). Leveraging plant physiological dynamics using physical reservoir computing. *Scientific Reports*, 12(1). <https://doi.org/https://doi.org/10.1038/s41598-022-16874-0>
- Pilati, D., Ceni, A., Michieletti, F., Gallicchio, C., Ricciardi, C., & Milano, G. (2026). Rcbench: A unified framework for benchmarking reservoir computing systems. *Neuromorphic Computing and Engineering*, 6(1), 014012. <https://doi.org/https://doi.org/10.1088/2634-4386/ae441f>
- Portland State University. (n.d.). Orca accelerator [Accessed: 2026-04-08].
- Rc bootcamp [Accessed 2026-02-03]. (2026). Retrieved February 3, 2026, from <https://rc-bootcamp.github.io/>
- Recht, B., Roelofs, R., Schmidt, L., & Shankar, V. (2019). Do imagenet classifiers generalize to imagenet? <https://arxiv.org/abs/1902.10811>
- Roy, O., & Vetterli, M. (2007a). The effective rank: A measure of effective dimensionality. *Proceedings of the 15th European Signal Processing Conference (EUSIPCO)*, 606–610.
- Roy, O., & Vetterli, M. (2007b). The effective rank: A measure of effective dimensionality. *European Signal Processing Conference*, 606–610. <https://doi.org/https://doi.org/10.5281/zenodo.40328>
- Sheffler, Z. M., Reddy, V., & Pillarisetty, L. S. (2025). Physiology, neurotransmitters [Updated 2023 May 1]. In StatPearls Publishing (Ed.), *Statpearls*. StatPearls Publishing. <https://www.ncbi.nlm.nih.gov/books/NBK539894/>
- Stepney, S. (2024). Physical reservoir computing: A tutorial. *Natural Computing*, 23(4), 665–685. <https://doi.org/10.1007/s11047-024-09997-y>
- Svensson, E., Apergis-Schoute, J., Burnstock, G., Nusbaum, M. P., Parker, D., & Schiöth, H. B. (2019). General principles of neuronal co-transmission: Insights from multiple model systems. *Frontiers in Neural Circuits*, 12, 117. <https://doi.org/10.3389/fncir.2018.00117>
- Tanis, J. E., Bellemer, A., Moresco, J. J., Forbush, B., & Koelle, M. R. (2009). The potassium chloride cotransporter kcc-2 coordinates development of inhibitory neurotransmission and synapse structure in caenorhabditis elegans. *Journal of Neuroscience*, 29(32), 9943–9954. <https://doi.org/https://doi.org/10.1523/jneurosci.1989-09.2009>
- Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature*, 393, 440–442. <https://doi.org/10.1038/30918>
- Wirak, G. S., Florman, J., Alkema, M. J., Connor, C. W., & Gabel, C. V. (2022). Age-associated changes to neuronal dynamics involve a disruption of excitatory/inhibitory balance in c. elegans (M. Zimmer & P. Sengupta, Eds.). *eLife*, 11, e72135. <https://doi.org/https://doi.org/10.7554/eLife.72135>

-
- Zheng, K., Qian, B., Li, S., Xiao, Y., Zhuang, W., & Ma, Q. (2020). Long-short term echo state network for time series prediction. *IEEE Access*, 8, 91961–91974. <https://doi.org/10.1109/ACCESS.2020.2994773>